



SDIO/EMMC CONTROLLER SPECIFICATION

Dan Gisselquist, Ph.D.
dgisselq (at) ieee.org

July 8, 2026

Copyright (C) 2026, Gisselquist Technology, LLC

This project is free software (firmware): you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <http://www.gnu.org/licenses/> for a copy.

Revision History

Rev.	Date	Author	Description
1.0	8/15/2023	Gisselquist	First release
1.9	2/24/2024	Gisselquist	Pre-release updates, with a new (optional) Wishbone DMA feature
1.91	4/25/2024	Gisselquist	New hardware reset feature added
1.92	7/22/2024	Gisselquist	New CRC token handling feature added
1.93	8/27/2024	Gisselquist	Updated token handling and DMA stream discussions
1.94	9/05/2024	Gisselquist	Note that ACKs should always be expected on block writes.
1.95	11/25/2024	Gisselquist	The design now fully supports AXI in addition to Wishbone, to include an (optional) AXI DMA
1.96	12/06/2024	Gisselquist	Added the 1.8V feedback wire
2.0pre	7/08/2026	Gisselquist	eMMC Boot support, and trim registers

Contents

	Page
1 Introduction	1
2 Features	2
2.1 Limitations	3
3 Architecture	5
3.1 Controller Architecture	5
3.2 DMA Architecture	8
3.3 eMMC BOOT Architecture	10
3.4 Verification Architecture	10
3.5 Software Driver Architecture	11
4 Operation	13
4.1 Waiting for a card	13
4.2 Issuing a Hardware Reset	14
4.3 Querying PHY Capability	14
4.4 Initial PHY setup	16
4.5 GO IDLE: Commands that don't get responses	17
4.6 SEND IF COND: R1 commands	18
4.7 ALL SEND CID: R2 commands	18
4.8 Adjusting the PHY	19
4.9 Reading a sector	20
4.10 Writing a sector	21
4.11 Writing multiple sectors	21
4.12 Reading multiple sectors	22
4.13 Using the DMA to write multiple sectors	24
4.14 Reading multiple sectors with the DMA	25
4.15 Using the DMA streaming interface to write multiple sectors	25
4.16 Reading multiple sectors to feed the DMA streaming interface	26
4.17 GO IRQ STATE	26
4.18 eMMC BOOT	27
5 Registers	30
5.1 CMD Register	30
5.2 Argument register	36
5.3 FIFO Registers	36
5.4 PHY Register	37
5.5 Soft Resets	40
5.6 DMA Usage	40
5.7 PHY TRIM	41
5.8 RX TRIM	42
6 Interrupts	44
7 Wishbone Datasheet	45
8 Clocks	47
9 I/O Ports	48

Figures

Figure		Page
3.1.	SDIO/eMMC Controller RTL Architecture	6
3.2.	SDIO/eMMC CMD pin data flow	7
3.3.	SDIO/eMMC transmit data flow, to the device	7
3.4.	DMA Block Diagram	9
3.5.	SDIO Verification IP	10
3.6.	Test-bench infrastructure block diagram	11
3.7.	Software Stack	12
5.1.	CMD Register fields	30
5.2.	PHY Front end configuration register	37
5.3.	DMA Stream config bit location for < 31b or greater address widths	41
5.4.	PHY Front end configuration register	41
5.5.	PHY Front end configuration register	42

Tables

Table		Page
5.1.	(Big-Endian) I/O Peripheral Registers	30
5.2.	CMD Register field descriptions, part one	31
5.3.	CMD Register field descriptions, part two	32
5.4.	Types of writes to the CMD register	34
5.5.	CMD response types	34
5.6.	CMD response types	35
5.7.	Potential receive error codes	36
5.8.	PHY Front end configuration register fields	38
5.9.	Width enumeration	39
5.10.	Clock division settings	39
5.11.	(Big-Endian) I/O Peripheral Registers	40
5.12.	PHY Front end configuration register fields	42
5.13.	PHY Front end configuration register fields	42
7.1.	Wishbone Slave Datasheet	45
7.2.	DMA Wishbone Master Datasheet	46
9.1.	List of IO ports	48
9.2.	AXI-Stream IO ports	49

Preface

After using the SDSPI companion controller for many years, I came across the need for an eMMC controller. Specifically, I needed a controller that could verify that two hardware interfaces were working. The first was a full SDIO interface, the second an eMMC interface,

Since the companion SDSPI IP already existed and was sufficiently low logic, this new IP took on a different set of goals. First and foremost, it was to support the full SDIO interface—all of it. Every timing mode was to be supported, as were all commands. This includes the multiple block read and write commands not supported by the SDSPI driver. This controller was also built to support eMMC devices—to include both the data strobe and the eMMC device interrupt. Unlike the SDSPI controller, low logic was no longer one of my goals.

Dan Gisselquist, Ph.D.

1.

Introduction

This SDIO/eMMC IP core is a redesign of the SDSPI IP, specifically focusing on full feature support. Low-logic, a key goal of the SDSPI companion IP, is no longer a goal. Key measures instead include full feature support and high device throughput. A full list of the features of this IP may be found in Chapt. 2.

For those interested in the basic architecture of how this controller is put together, Chapt. 3 describes its basic components and provides illustrations of how those components work together to accomplish the various actions required of the IP. Chapt. 3 will also discuss the components of the verification architecture.

I am aware that there is an official SDIO controller register set standard. This IP does not support that standard, but rather uses its own register set configuration. One reason for this is simplicity. The official register set standard appeared, upon inspection, more complicated than necessary. Second, since the SDSPI register set worked so well, there didn't seem to be a need to generate something new.

As a result, most of the registers from the SDSPI controller have been replicated here, save that they support SDIO control instead of SPI control. Like the SDSPI controller, this IP has CMD, DATA/ARG, and two FIFO registers. Unlike the SDSPI controller, a fifth register has been added for PHY configuration. This register directly exposes what once was the internal configuration register of the SDSPI controller. Finally, this IP now supports an optional internal DMA capability, and so three additional registers are now assigned to support both the DMA address and transfer length—registers not present in the original SDSPI controller. Chapt 4 discusses how all of these registers can be used to accomplish the basic operations required of any interaction with an external card/chip, while Chapt. 5 goes through the definition of each register in detail.

Finally, this is an Open Source project. The project was initially developed using a combination of external funding and internal research and development dollars. My intention is twofold. First, I intend to use this project to both teach others, as well as for blogging material. From this standpoint, the project exists as part of a portfolio of projects available for others to examine and learn from. My second goal is to use this as a background when providing services to my customers. My typical contract is for an hourly rate only, yet it comes with full access to any IP I have developed. As such, additional work on this IP may be done on a pay-per-hour basis. Where this project supports another project of mine, I will bill that project for any required maintenance. Likewise, if you find this IP almost meets your needs, then feel free to contract with me for any maintenance adjustments or upgrades you might need.

2.

Features

This IP has been designed from the ground up for full SDIO/eMMC feature support. This means that support has been built in for the following specific requirements:

- Full IO support
 - Support for both open-drain and push-pull IOs are available.
 - Both SDR and DDR modes are supported, for clocks up to 2x the system clock.
 - IP supports either 1, 4, or 8 bit bus widths
 - The eMMC data strobe is supported
- Supports both multiblock read and multiblock write commands.

These are commands that were not supported by the SDSPI IP. They are supported here, at the cost of additional logic, since the goals of this project include both full feature support and high device throughput. Therefore, these features are supported at the cost of more logic.
- Ping-pong FIFO support, so one FIFO may be active on the bus while the other is active in a transaction.
- Internal CRC generation and checking, for both command and data interfaces
- Both Command response and receive block timeout support. Timeouts may be adjusted at build time if necessary.
- (Optional) Card detection support
- Integer clock division support starts at 2x the system clock rate, and can divide the clock all the way down to 1/1,000th of the system clock rate. Thus, for a 100 MHz system clock, clock frequencies supported include 200 MHz, 100 MHz, 50 MHz, 25 MHz, 12.5 MHz, 10 MHz, 8.3 MHz, etc., all the way down to 100 kHz.
- The SDIO clock may be stopped upon request, once all operations have come to a stop. This is useful when working with the multiblock read command, as the clock may be stopped while data is transferred out of the FIFOs. The feature can also be used to minimize emissions and power usage when the clock isn't in use.
- Special support for eMMC's *GO_IRQ_STATE* command. Receive timeout detection is suspended for this command. Further, it is possible to self-send a reply to this command to exit interrupt mode if the device doesn't generate an interrupt fast enough.

- Includes software driver support for use with [FATFS](#)
- Both Verilator C++ and all Verilog device verification models are included
- While the IP is primarily built and tested for big-endian buses, little endian support exists as a top level parameter. This is useful when working, for example, with AXI buses which are required to be little endian.
- All of the various IP modules, save for the front end itself, can be formally verified.
- 32-bit Wishbone control interface.

An optional AXI-Lite control interface also exists.

- An optional DMA interface exists. This DMA interface can be used to transfer many blocks of data sequentially as part of a single command.

The DMA interface can master either Wishbone or AXI4 buses, and it is not limited to 32b bus widths.

Many, although not all, of these features have been tested as part of the [10Gb Ethernet switch KlusterLab project](#). It is in this project that the software has been tested and proven, and where its utility has been demonstrated. The SD card component of the design has since been ported to my [VideoZip](#) repository, where further testing continues to take place.

2.1 Limitations

As of this current release, the IP is not as fully featured as I might like. Several details and features are missing and may (or may not) be developed in the near future as time, funding, and necessity require. The following list, therefore, outlines some of the key limitations of the controller as it exists today. It also provides a roadmap for future development.

- IO features not yet tested in hardware include: data strobe support. To date, these feature(s) have only been tested either formally or in simulation or both.

This is due to both the voltage requirements of these extra features, as well as the design of the circuit board I am currently testing with. High speed support requires 1.8V not 3.3V, and (worse) often requires a voltage switch from 3.3V to 1.8V. Second, the voltage translator we are using limits speeds to 60MHz in order to maintain the ability to support open-drain IO. Similarly, wiring the clock pin to Xilinx's CCLK pin has limited the clock in the devices under test to a maximum of 50 MHz. Finally, success with the data strobe pin really requires that the data strobe be wired to a clock capable pin. I expect these hardware testing limitations will be addressed in the future with a better PCB design, but for the time being this statement outlines hardware testing success to date.

Please check the [hwteststat.png](#) file in the ./ directory for the current status of any hardware testing and any updates on this issue.

- The Verilog verification model depends upon an internal memory buffer which can become quite large. Further, it doesn't integrate nicely with the FatFS system.

A C++ verification model now exists for the SD card model. This model allows for modeling SD cards of arbitrary sizes, making it possible to test file system handling. There is no eMMC version of this model yet.

- The included verification test suite doesn't (yet) support testing the software driver. Software verification has instead taken place as part of other projects.
- While the IP has been designed to support all commands, the verification model doesn't (yet) support all SDIO commands.

As additional commands are required, they will be added. At present, however, the SDIO and eMMC models are both sufficient to verify IP startup and basic data transfer. In addition, the eMMC model is also sufficient to verify the eMMC BOOT capability.

3.

Architecture

Four basic architectures are discussed in this chapter. The first is the hardware architecture. This will outline the various hardware components composing the design, and discuss each of their purposes. This is followed by a discussion of the optional DMA architecture component of the overall hardware architecture. The third is the verification architecture, used to test and verify the various components of the design. Finally, this chapter will conclude with a quick review of the software driver architecture that allows a single [FATFS](#) library to be able to interact with all three of the design components here: SDSPI, SDIO, and eMMC.

3.1 Controller Architecture

The basic SDIO/eMMC RTL architecture is shown in Fig. 3.1. The design is broken up into two parts at the top level: a hardware independent, all digital logic component whose top level is [sdio.v](#), and a hardware dependent front end component whose top level is found in [sdf frontend.v](#). These two are combined together in a top level SDIO component, [sdio_top.v](#).

The core controller, found in [sdio.v](#), itself is composed of five separate components. The first, [sdwb.v](#), is the controller. This component is a Wishbone slave. All user interaction goes through this controller. Requests for transfers are made here. Data to be transferred is stored here in one of two FIFOs for transmission to the front end. Data to be received is stored here after reception, to be read out as it becomes available to the CPU. Even when using the DMA, data will first be stored in the FIFO's within this controller before going to the card, or after coming from the card.

A second, alternative, controller implementation exists in [sdaxil.v](#). This controller is nearly line for line identical to the [sdwb.v](#) controller, save that interaction with this controller takes place over AXI-Lite instead of Wishbone.

The controller maintains a PHY register for configuring the front end. This register also controls the clock generator, [sdckgen.v](#). This clock generator is responsible for generating the clock used throughout the interface. That clock is internally represented as an 8-bit vector, allowing it to specify zero, one, or up to two clock periods, each either aligned with or 90 degrees offset from the data, per 8-bits. This clock may also be divided to slower speeds as requested by the user. Finally, a user setting allows the clock to turn off when it isn't being used, to save on interface power and to potentially reduce emissions. (Note that this only turns off the IO clock, not any of the system clocks used throughout the design to clock the logic within the design.) Turning off the clock may be required when working with the multiblock read command, to ensure that the FIFO is properly cleared before subsequent blocks are read.

Commands issued to the controller will be forwarded to the [sdcmd.v](#) module. This module is responsible for all interactions taking place over the CMD wire. The module can generate 48-bit

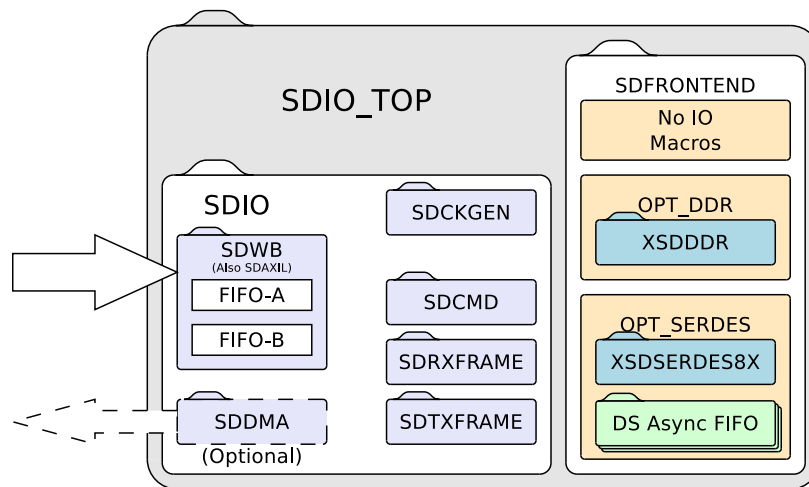


Figure 3.1: SDIO/eMMC Controller RTL Architecture

commands, beginning with a start bit and ending with a CRC and a stop bit. It can then recognize either 48-bit or 136-bit command responses. Responses are checked for potential CRC or frame errors. If the device fails to return a response within a given timeout window, the response may also timeout with an error, after which the command controller will no longer expect any response until the next command is sent.

Fig. 3.2 shows both the basic command flow, and the two possible response flows. For commands, a data register within `sdwb.v` is first set, then the command is issued. The `sdcmd.v` module organizes both into a 48-bit word to be transmitted. The `sdcmd.v` module then looks for one of two responses, either a 48-bit response which would place the 32-bit argument back in a data register, or a 136-bit response which would be placed into one of the two FIFOs in `sdwb.v`.

One type of controller command is a memory command. These commands send a 48-bit command to the device, followed by sending data to or receiving data from the device.

The `sdrxframe.v` component is activated when data is expected from the device on the data wires. Its purpose is to collect the data returned from the front end controller, and re-form that data into commands to write to one of the FIFOs in `sdwb.v`. If a full frame, as defined by the controller, is not received prior to a timeout, or if a CRC error is detected within that frame, the receive component will report an error and give up in order to wait for its next command.

Similarly, the `sdtxframe.v` command is activated when data is to be sent over the data wires to the device. In this case, data flows from one of the FIFOs in `sdwb.v` to `sdtxframe.v`, gets formatted for the front end to output, a start bit, CRC, and stop bit are added, and all get sent to the front end. This basic flow is shown in Fig. 3.3. The `sdtxframe.v` transmit component can also be configured to expect and wait for a CRC token following a successful transfer.

Let's now turn our attention to the device dependent front end.

The front end was initially defined for full IO support, all the way up to HS400 and the eMMC data strobe. Driven from a 100 MHz clock, this means that this front end was designed to support up to two device clock periods per 100 MHz system clock, and to have data sent on each edge of

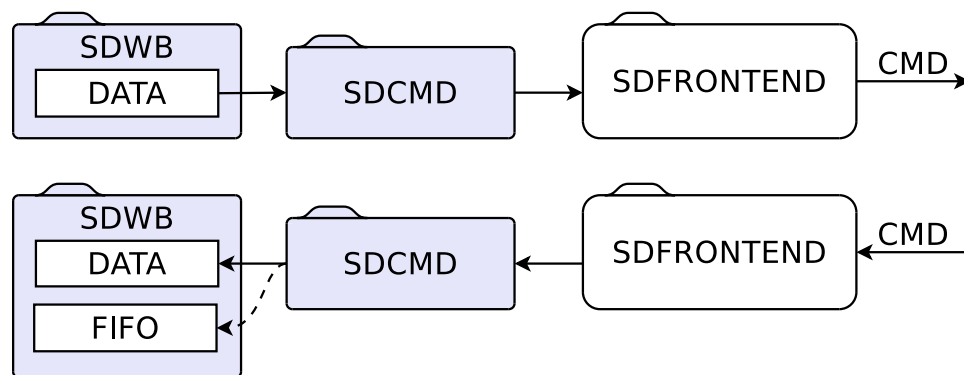


Figure 3.2: SDIO/eMMC CMD pin data flow

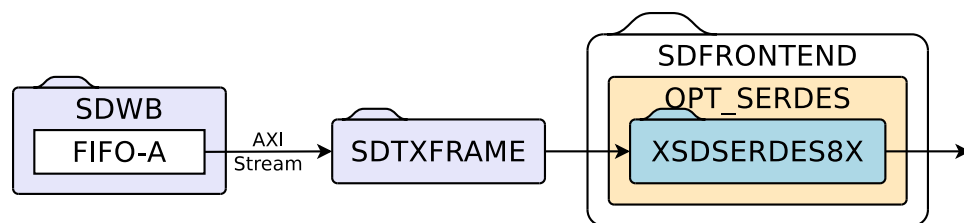


Figure 3.3: SDIO/eMMC transmit data flow, to the device

these two clock periods. This forces a requirement of 4-data bits output per 100 MHz system clock cycle per data pin. The clock, however, also needs to be (potentially) offset by 90 degrees from that data. As a result, all I/O using this particular front end capability, called the `OPT_SERDES` capability in Fig. 3.1, takes place using Xilinx's 8:1 OSERDES elements. Incoming bits are either sent to an asynchronous FIFO, to support the eMMC data strobe, or they are sampled via 1:8 ISERDES elements. In the case of the ISERDES inputs, sample timing is controlled via both the output clock rate, as well as a user controlled digital delay, allowing sample alignment with clock precision of one eighth of a system clock. Hence, even without the full speed HS200 support, this subsample re-alignment may still be quite valuable when supporting higher speeds. All Xilinx specific SERDES interactions are captured in an `xsdserdes8x.v` module.

Logic resources throughout the design may be reduced, however, by switching to a simpler front end architecture. Two other architectures are available: the `OPT_DDR` architecture and a plain architecture that doesn't use any IO macros—save for the `IIOBUF` macro used for controlling any tristate capabilities.

The `OPT_DDR` architecture is roughly the same as the `OPT_SERDES` architecture, save the DDR IO elements are used to generate both clock and data instead of the SERDES elements. This should make it possible to drive the IO pins at the full clock speed using SDR, or at half the clock speed using DDR. In this case, all Xilinx specific IDDR and ODDR interactions are captured in an `xsdddr.v` module.

The no macro architecture was built in order to support an architecture where the `SD_CK` pin was routed through the Xilinx `CCK` pin, and hence through an `STARTUPE2` primitive. In this case, no IO macros are available for controlling the clock, and all IOs are controlled directly from logic. The fastest clock this architecture can generate is half the system clock rate.

These constitute the basic components of the controller, both hardware independent and hardware dependent.

3.2 DMA Architecture

The DMA is an add-on to the basic architecture, and as a result is an optional feature of the design. It is built from several parts. The first key component to the DMA is found in the control module, `sdwb.v`. When the DMA is present, the control module becomes responsible for sectioning transfers into per-block requests, and forwarding data to and from the DMA module itself. If buffers fill while reading, it will automatically stop the clock. Once all data has been read, the control module will issue a `CMD12` (`STOP_TRANSMISSION`) to end the transfer.¹ The rest of the DMA logic is found in the DMA module, `sddma.v`. The structure of this module is shown in Fig. 3.4.

When using the DMA, data continues to go through the FIFOs in the controller module just as it would without the DMA present. The difference is that these FIFOs will be automatically sourced from, or emptied into, a DMA module which will then handle copies from, or to, memory. Hence, for a write operation, the user will request the write operation and then the controller will tell the DMA to supply data for one of its FIFOs. Once the FIFO is filled, the controller will send it to the external device. Then, while the data is being transmitted to the device, the controller will continue to request source data capture to supply any subsequent data transfer requirements. Likewise, for a read operation, the user will request the operation first, followed by a block getting transferred

¹When using the DMA to perform an alternative eMMC boot sequence, the transfer will end with a `CMD0` (`GO_IDLE`).

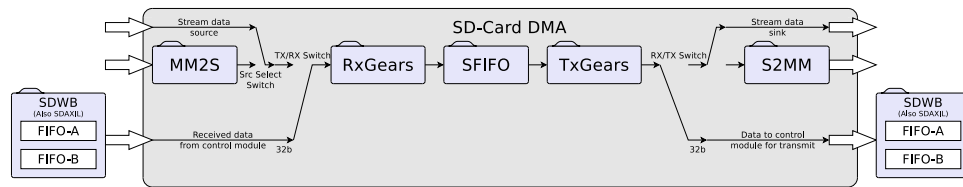


Figure 3.4: DMA Block Diagram

to the controller's FIFO. The read will then continue filling the second FIFO will the first one is drained by the DMA. If both FIFO's fill faster than the DMA can empty them when reading from the device, then the IO clock will be stopped to wait for the next FIFO to be drained.

That's a high level overview, from the perspective of a DMA enabled control module. Let's now turn our attention to how the DMA supplies and deals with the data streams given to it.

Data arrives in the DMA module from one of three sources. First, if both `OPT_ISTREAM` and the MSB of the DMA address are set, data may arrive from an AXI stream. If used, this most-significant bit is one wider than the address width of the devices bus. Alternatively, data may be read from memory using the `sddma_mm2s.v` (or `sdax_mm2s.v`) component. Finally, when reading from the card, data will arrive first in the controller module, and then be forwarded from it once the respective buffer is full.

These sources may have different data widths. Data arriving from memory will arrive packed in bits that may be up to the full bus width in size. Data arriving from the controller will always be 32b wide. Data arriving from the stream port may be either of these widths. The receiving gearbox, `sddma_rxgears.v`, will therefore take this data stream and pack it into beats at the widest of these widths.

From here, data enters into a FIFO. This guarantees that the memory source never needs to stall, or otherwise provide backpressure on the bus—something not supported by the Wishbone interface. The FIFO also allows data to arrive at wide widths, only to be read at slower (32b) rates if necessary.

From the FIFO, the data width is adjusted in the transmit, or outgoing, gearbox, `sddma_txgears.v`. This component optionally adjusts the data width back down to 32b if required. Either the stream output, or the controller interface may require this width. If going back to memory, the width will be left at the full size of the bus.

The final step in the DMA process is to forward the data stream to one of three ultimate destinations. When writing to the card, the ultimate destination will send the data to the controller. When reading from the card, data may either be written to memory or to the AXI stream interface. The AXI stream interface will be used when/if both `OPT_OSTREAM` is set and the most significant address bit are set.

A small amount of logic exists in the `sddma.v` module to control the switching between sources and outputs, as well as to make sure block boundaries are respected.

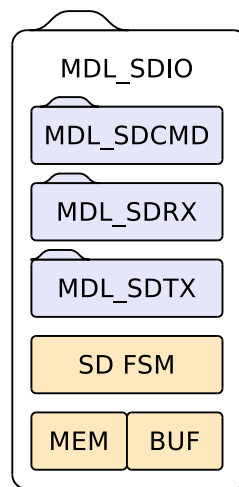


Figure 3.5: SDIO Verification IP

3.3 eMMC BOOT Architecture

A second optional addition to the controller architecture is the eMMC BOOT capability. This additional capability is primarily handled by an upgrade to the control module, whether `sdwb.v` or `sdaxil.v`. This additional logic is enabled by setting both `OPT_EMMC` and `OPT_BOOTEN`.

This logic can be used to hold the command wire low (CMD), wait for a device acknowledgment, and then transfer block after block of data to memory—either via the DMA or (with CPU/User interaction) from the FIFOs.

3.4 Verification Architecture

The verification architecture is built around a model of the downstream device. One such model is shown in Fig. 3.5. A second, similar, Verilog model exists for modeling the eMMC interface. These Verification IP modules contain full featured submodules for handling the command wire, receiving data, and transmitting data. These three submodules are designed to correspond to their host controller counterparts. Further, these three submodules have been built to be independent of whether the controller is modeling an SDIO or an eMMC interface.

The smarts of this Verification IP (VIP) are found in the finite state machine (FSM) at the top level, in either `mdl_sdio.v` or `mdl_emmc.v`. This state machine responds to commands, and generates responses to them. A data transfer buffer exists for sending or receiving data across the interface. A second memory within the VIP, shown as MEM in Fig. 3.5, captures the persistent memory found in the SDIO (or eMMC) device.

The entire Verilog test bench infrastructure is diagrammed in Fig. 3.6. In this setup, a test script drives a Wishbone Bus Functional Model (BFM). The BFM issues 32b bus requests, which are then resized to the full width of the bus. From here, requests go into a crossbar, and get routed to either an SDIO (via a width downsizer), an eMMC controller (via a width downsizer), a block

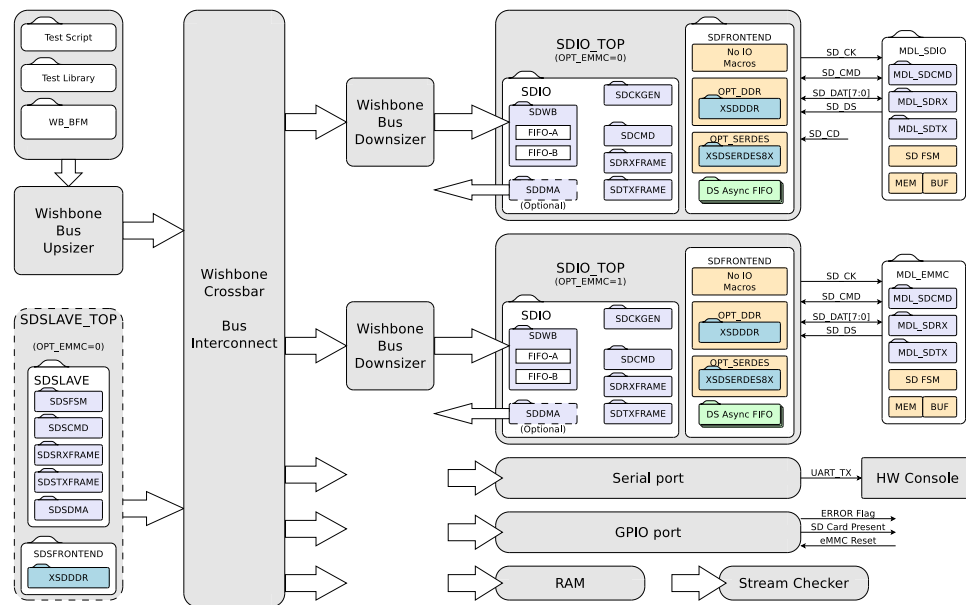


Figure 3.6: Test-bench infrastructure block diagram

RAM model (at the full bus width), a console port, a GPIO port, or a stream checking controller. This then allows the test script to interact with the IP, which then drives interactions with one of the two backend VIP device models. If the DMA's are present, they can also be used to transfer data to or from the RAM via the bus interconnect. If the stream option has been enabled, the stream checker can be used to both generate stream data and to verify that incoming data matches what had been generated. What tests get accomplished at this point is then dependent upon the test script. Two test libraries are also available, [one for SD cards](#) and [one for eMMC cards](#). These libraries help define common constants and supporting common test script tasks which can be called by the test script itself.

This test bench also supports the SDSLAVE component, discussed elsewhere. This component is a slave to the SDIO bus. It is a separate product, though also found and released in this repository. If present, this component will replace the SDIO model, [mdl_sdio.v](#). By placing it in the top level test bench, the SDSLAVE component may be tested as a slave to the SDIO controller, if desired, using many of the same test scripts as are used to test the SDIO controller itself.

Two versions of this top-level interface exist. The first supports all Wishbone interactions. It can be found in [tb.wb.v](#). A second top level exists for testing AXI interactions. This top level test bench may be found in [tb_axi.v](#).

3.5 Software Driver Architecture

At present, there exist three software drivers and one glue layer for each. The three drivers are found in the [sw/](#) directory. These respective drivers are:

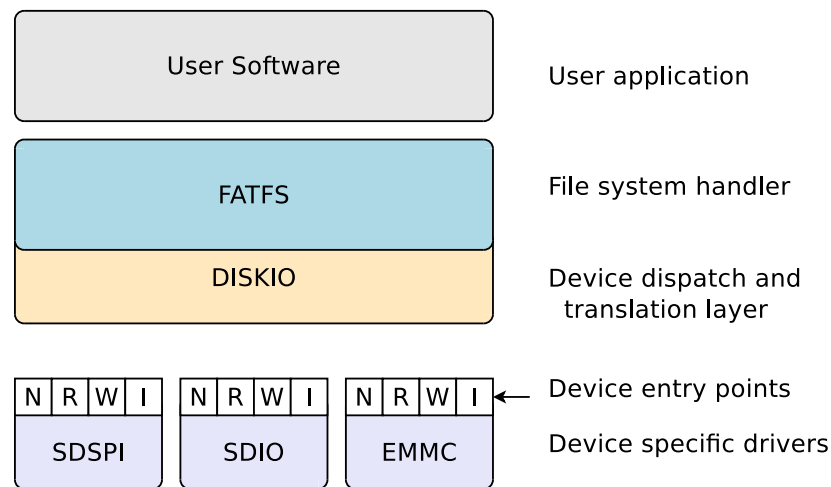


Figure 3.7: Software Stack

- [sdspidrv.c](#): A software driver for the SPI version of this controller. The SPI version of this controller is described in the SDSPI user guide.
- [sdiodrv.c](#): A software driver for the SDIO controller, described in this user guide. This controller is designed to interact with an SD Card.
- [emmcdrv.c](#): A software driver for the eMMC controller. This is designed to interact with the same SDIO/eMMC controller discussed in this user guide. However, the SDIO and eMMC protocols require different commands to interact with their respective devices. As a result, in spite of any similarities, the eMMC software driver differs significantly from the SDIO software driver.
- [diskiodrvr.h](#) provides a common definition of a device “driver”, which can be used across all of the device drivers listed above.

More on this in a moment.

- [diskio.c](#) is a shared/common dispatch glue layer. Commands from the [FATFS](#) subsystem naturally land here, where they are then dispatched to the appropriate controller.

Put together, this IO subsystem has the various layers shown in Fig. 3.7. User software interacts with the [FATFS](#) layer. [FATFS](#) then interacts with the device dispatch layer. The device dispatch layer then instantiates and interacts with one (or more) of the device drivers beneath it.

Each device driver has four entry points—“methods” in Object Oriented terminology. The first method is used to initialize the driver and set up its data structures. After that, there are methods for reading and writing. Finally, there’s an IOCTL method which can be used for other tasks such as querying the size of the device or the size of a data block.

4.

Operation

At its most basic level, interacting with the controller simply involves configuring the PHY, setting the data you wish to send, and then sending a command to the controller. This needs to take place, however, in many different contexts. Some commands don't expect responses, whereas others return 48 or even 136-bits. Some commands are followed by data transmission, whereas others do not. This section therefore walks through several examples of commands, illustrating the range of commands and responses that can be issued using this controller.

4.1 Waiting for a card

Prior to a card being inserted in the slot, the design will be held in reset. Hence the first important step when working with an SD card is to wait for the SD card to be present.

```
const unsigned SDIO_PRESENTN = 0x00080000;
unsigned cmd;

do {
    cmd = _sdio->sd_cmd;
} while (cmd & SDIO_PRESENTN);
```

Once an SD card is present, as part of the initialization process, the first step is then to acknowledge it.

```
const unsigned SDIO_NULLCMD = 0x00000080,
SDIO_REMOVED = 0x00040000;

_sdio->sd_cmd = SDIO_REMOVED | SDIO_NULLCMD;
```

Only after clearing the SDIO_REMOVED flag may additional work take place.

If SDIO_REMOVED bit is ever set from here on out, it will indicate that the SD card has been removed. If the command register indicates a card is present, yet the removed flag is still high, it may be that a card was removed and a new one inserted. Either way, the controller will need to restart the card with a GO_IDLE command.

4.2 Issuing a Hardware Reset

Unlike the SDIO interface, the eMMC interface often includes a hardware reset pin. This pin is controlled from the same control register. To enable the reset, one need only write to the command register with the reset bit present.

```
const    unsigned    EMMCHWRESET = 0x02000000;
unsigned    cmd;

// Enable the reset
_sdio->sd_cmd = EMMCHWRESET;

// Disable the reset
_sdio->sd_cmd = SDIO_NULLCMD;

do {
    cmd = _sdio->sd_cmd;
} while (cmd & EMMCHWRESET);
```

Writing a NULL command, SDIO_NULLCMD, can be used to clear the reset indication.

Beware when using this reset that eMMC has a minimum reset length requirement. Hence, just because the reset has been commanded to be cleared doesn't mean that it has been cleared yet. For this reason, the reset bit of the command register may then be polled to know if and when the device has completed its reset.

While SD cards don't necessarily have a reset pin, there are reasons to control the power to an SD Card. One such reason might be to enable the card to transition back from 1.8 V to 3.3 V modes. The hardware reset pin used in eMMC may also be used to control a power pin if so desired.

4.3 Querying PHY Capability

It can be important to know how the SDIO controller was configured, so let's continue there next. Specifically, let's start by trying to determine which version of the PHY is present. We can do this by requesting a 200 MHz clock, an 8 bit bus width, DDR mode, and a full PHY shift. We'll also allow the device to turn the clock off when not in use, so we that we don't push a high speed clock accidentally before the device has been properly configured for it.

```
const    unsigned    SDIOCK_200MHZ = 0x00000000 ,
SDPHY_W4      = 0x00000400 ,
SDPHY_WTEST   = 0x00000c00 ,
SDPHY_DS      = 0x00004300 ,
SDPHY_DDR     = 0x00004100 ,
SPEED_CLKOFF  = 0x00008000 ,
SDPHY_SHFTMSK = 0x001f0000 ,
SECTOR_MAX    = 0x0f000000 ;

unsigned    phy;

_sdio->sd_phy = SDIOCK_200MHZ | SDPHY_WTEST | SDPHY_DDR
                | SDPHY_SHFTMSK | SPEED_CLKOFF | SECTOR_MAX;
```

```
while(3 < (phy = _sdio->sd_phy & 0x0ff))
;
```

As always, whenever changing the clock speed, it's important to wait for the change to complete. Notice, though, that we aren't waiting for the clock to reach 200 MHz. Instead, we wait until the clock is at least 25 MHz. This is because not all front ends can support the 200 MHz clock. Therefore, we wait for the clock rate to change, and then see which clock rate we have.

We can now examine the **phy** return value to learn our configuration.

- If `0 == (phy & SDPHY_WTEST)`, then we know the controller will only support a single data bit. If `SDPHY_W4 == (phy & SDPHY_WTEST)`, then we know it will support either 1 or 4 data bits. Otherwise, the PHY will support an 8 bit interface—useful for eMMC devices.
- The **SDPHY_SHFTMSK** specifies 5 bits of a shift register which can be used to control where return sampling takes place. If all bits remain set in **phy**, then design was built with the SERDES front end. If all but the bottom two bits are set, then the DDR front end has been built, etc.

Depending upon which version of the PHY is available, the sample shift may need to be adjusted. Beware, though, that this may be a board specific setting.

```
unsigned        sample_shift = 0;

// Shift amounts shown are only notional, and may (or may
// not) reflect the appropriate values for your hardware.
if (phy & 1) {
    // SERDES PHY
    sample_shift = 9 << 16;
} else if (phy & 4) {
    // DDR PHY
    sample_shift = 16 << 16;
} else
    sample_shift = 24 << 16;
```

- The clock frequency returned indicates the fastest clock rate the design can operate in DDR mode. Often, the design can go one speed step faster in a non-DDR mode, but this is still useful as a starting point.

For example, the DDR PHY can handle SDR104 (a 100MHz clock) when operated from a 100MHz system clock. This would have a clock setting of `1 == (phy & 0x0ff)`. However, DDR requires a 90 degree clock offset from the data, and achieving this offset will force the maximum speed to be `2 == (phy & 0x0ff)` (50 MHz or DDR50).

- If the DS bit is set, `(phy & 0x0200)`, then the PHY can support return data strobe handling.
- Finally, the PHY register will indicate the maximum sector size available to it—typically 512 Bytes, but it might be larger. This size is given in bytes by the expression:

```
sz = (1 << ((phy >> 28) & 0x0f));
```

A similar approach may be used to determine if the DMA is present. We can start by writing a negative one to the length register, and then reading the result back.

```

unsigned      ln;
char         *addr;

_sdio->sd_dma_length = -1;
ln = _sdio->sd_dma_length;
if (ln != 0) { // DMA is present

```

If the return value is non-zero, then the DMA is present.

To find out whether or not the stream capability is present, write a negative one to the DMA address register.

```

char         *addr;

addr = (void *)-1;
_sdio->sd_dma_addr = addr;
addr = _sdio->sd_dma_addr;
if (((unsigned)addr) & 1) {
    // No stream capability
    // addr is the available address mask
}

```

If the stream capability is present, this will select the stream bit and all other bits will be zero. The result minus one will yield a bit mask indicate how many valid address bits are available on the bus. If no stream capability is available, then the return value by itself will indicate the number of valid address bits on the bus.

The software controller may also be tuned for a particular architecture. In that case, these steps may be skipped. They are really only required when one software driver needs to interact with devices of potentially varying capability.

Note that these tests, and any initial PHY setup, may still be accomplished even if the eMMC hardware reset wire is active (low).

4.4 Initial PHY setup

The first step to any operation is to make sure that the PHY register is appropriately set up for the clock speed desired, the transfer rate desired, and whether or not the design will be using open-drain or push-pull IOs. Any changes made to this register will be applied immediately, save for any changes to the clock rate. Clock rate changes will take place starting at the top of the next clock cycle. Hence, once a clock rate change has been made, it makes sense to wait until the current clock matches the requested clock rate.

The following, for example, will set up a 100 kHz clock rate using open-drain IOs and 512 byte sector sizes. This is appropriate for beginning communication with the device.

```

const    SDIOCK_100KHZ = 0x000000fc ,
          SPEED_SLOW    = SDIOCK_100KHZ,
          SPEED_512B    = 0x09000000 ;

```

```

_sdio->sd_phy = SPEED.SLOW | SECTOR_512B | sample_shift;
while(SPEED.SLOW != (_sdio->sd_phy & 0xff))
;

```

Note that one is required to wait for the clock rate to match. When coming from a faster speed, the clock rate change may appear instantaneous. When coming from a slower speed, this wait may require several clock cycles.

4.5 GO IDLE: Commands that don't get responses

Sending a command to the device is as easy as setting the command's 32-bit argument, and then writing the appropriate command to the command register. All command writes set bits [7:6] to 2'b01. Non-command writes will set these bits to something different. At the same time you write a command to the controller, you'll also want to tell the controller what type of response to expect—whether to expect no response at all, a 48-bit (R1) response, a 48-bit response followed by a busy (R1b), or a 136-bit (R2) response.

When issuing a command, you also want to be aware of the error bit. If the error bit gets set, no commands will be accepted until the error bit is cleared. Hence, you'll want to clear the error bit at the beginning of any command sequence, and check it again when you are done to see if and where any error took place.

Finally, in the case of the first command used when interacting with a device, you'll want to clear the controller's card-removed bit—if you haven't done so before. You can then check this bit later, at your convenience, to know if the card has ever been subsequently removed and (if so) if you need to start your initialization sequence again from the SEND_GO_IDLE command.

Put together, this would look like:

```

const    SDIO_REMOVED = 0x00040000,
          SDIO_ERR     = 0x00008000,
          SDIO_RNONE   = 0x00000000,
          SDIO_CMD     = 0x00000040;

_sdio->sd_data = 0;
_sdio->sd_cmd  = SDIO_REMOVED | SDIO_CMD | SDIO_RNONE | SDIO_ERR;

```

The next step is to wait for the command to complete. This can be done either by polling the controller,

```

const    SDIO_BUSY    = 0x00106800;

unsigned    st;

st = _sdio->sd_cmd;
while(st & SDIO_BUSY)
    st = _sdio->sd_cmd;

```

or by waiting for a controller interrupt.

```

// SDIO_INT is the interrupt number assigned to the
// design. It's set external to the controller.
unsigned      st;

st = _sdio->sd_cmd;
while(st & SDIO_BUSY) {
    CLEAR_INT(SDIO_INT);    // Clear the interrupt signal
    WAIT_INT(SDIO_INT);     // Wait for the given interrupt
}

```

4.6 SEND IF COND: R1 commands

The next type of command the controller understands are what are called R1 commands. These are commands that expect a 48-bit response—whether or not that response truly contains an R1 register or any other register.

The first R1 command in the SD card startup script is the SEND IF COND command, also known as CMD8. This command requires a special argument. We'll allow that argument to arrive via the variable `ifcond`. This argument is first placed in the `sd.data` register, and then the command is issued.

```

const    SDIO_R1      = 0x00000100,
          SDIO_READREG = SDIO_R1 | SDIO_CMD;

_sdio->sd_data = ifcond;
_sdio->sd_cmd = SDIO_READREG+8;

wait_while_busy();

```

Once the command returns, the first 8-bits of the 48-bit return may be read from the `sd_cmd` register. If the protocol has been followed and the device has returned a value, then bits [7:6] of this register will now be zero. The next 32-bits may be read from the `sd_data` register, to know how the device has responded. The last 8-bits, both CRC and stop bit, are not returned by the controller. Instead, the error bit and error code may be checked to know if the CRC is correct and the stop bit valid.

4.7 ALL SEND CID: R2 commands

The last type of command is the R2 command. These commands read 136-bits from the device. The first 8-bits of these are read into the command register, as with the R1 commands. However, the next 128-bits will not fit into the data register. Instead, they are returned into one of the two FIFOs.

Sending such a command follows the same script as before. The difference is that we could now specify, in our command, which FIFO to write the results into. We'll use FIFO A, the first FIFO, which is the default for this example.

```

const    SDIO_R2      = 0x00000200,
          SDIO_READREG = SDIO_R2 | SDIO_CMD;

_sdio->sd_data = 0;
_sdio->sd_cmd = (SDIO_ERR|SDIO_READR2) + 2;

wait_while_busy();

```

If this particular command works, it is likely to set the CRC error bit. This is normal for a CMD2, although not so normal for other commands.

Once the command completes, the CID register can be read from the FIFO 32-bits at a time.

```

unsigned    d_CID[4];

d_CID[0] = _sdio->sd_fifa;
d_CID[1] = _sdio->sd_fifa;
d_CID[2] = _sdio->sd_fifa;
d_CID[3] = _sdio->sd_fifa;

```

If the controller has been configured for big-endian operation, then the MSB byte of each FIFO result will be the first byte read from the device.

The d_CID values may then be decoded and processed by the software driver.

4.8 Adjusting the PHY

Once any cards have been properly enumerated, the PHY may be adjusted for higher speed. The first speed change below simply changes the clock speed to 25MHz, and the IOs from open-drain to push-pull.

```

const    SDIOCK_25MHZ = 0x00000003,
          SDIO_PUSH_PULL = 0x00000300,
          SPECTOR_512B = 0x09000000;

_sdio->sd_phy = SPECTOR_512B | SDIOCK_25MHZ | SDIO_PUSH_PULL
               | sample_shift;

```

Once we request a clock change, we may then need to wait a while. Specifically, the clock will only change in between clock periods. Hence, if the clock was set for 100 kHz operation, we may need to wait up to 10 μ s for the clock to switch speeds. We can tell when the switch is complete by reading back the lower 8-bits from the PHY register to know the currently set clock speed.

```

// Wait for the clock change speeds
while(SDIOCK_25MHz != (_sdio->sd_phy & 0xff))
;

```

A later PHY adjustment might even include switching to 4-bit data mode.

```

const    SDIO_W4 = 0x00000400;

```

```
_sdio->sd_phy = SECTOR_512B | SDIOCK_25MHZ | SDIO_PUSH_PULL  
| SDIO_W4 | sample_shift;
```

In this final example, we don't need to wait for the change to take place. The interface was already idle, so it took place immediately.

4.9 Reading a sector

Once the SD card has been set up, the next step may be to read a sector of data. In general, this is as simple as enabling the FIFO when issuing the command, and then reading the data from it once the command has been complete.

The SDIO driver first, however, checks whether or not the card has been removed. This is appropriate, lest the card has been swapped unbeknownst to the controller.

```
if _sdio->sd_cmd & SDIO_REMOVED)  
    return CARD_REMOVED_ERR;
```

If the card has not been removed, then we can issue a command to read a block. This involves setting the SDIO_MEM bit and (potentially) the FIFO bit to select which FIFO we are interested in.

```
const    SDIO_MEM      = 0x00000800,  
          SDIO_READBLK = (SDIO_CMD|SDIO_R1|SDIO_MEM)+17;  
  
_sdio->sd_data = sector;           // Assumes a high capacity card  
_sdio->sd_cmd = SDIO_READBLK;  
  
wait_while_busy();
```

Once the read completes, we can check for any errors.

Assuming the read completed without error, we can then read the data back out of the controller.

```
for(int k=0; k<512/sizeof(uint32_t); k++)  
    buf[k] = _sdio->sd_fifa;
```

Note that the FIFO's are always 32-bit aligned. Whether or not the buffer is aligned, or whether or not the CPU supports unaligned accesses in the case where it isn't, aren't controller issues.

If no DMA is used, it will likely cost the CPU 128 memory load cycles, followed by another 128 memory store cycles to read this data out.

An external DMA, such as the ZipCPU's DMA, may also be used to read from this controller if desired for higher speed. In this case, the read will cost 128 clock cycles (plus latency) to read the memory out of the controller, and another cycles equivalent to 512 divided by the bus width to write the result to memory.

4.10 Writing a sector

Writing a sector to the device is very similar to reading, however, there are a few key differences. The first key difference is that the memory needs to be loaded into the FIFO first, before it can be sent to the device.

```
for(int k=0; k<512/sizeof(uint32_t); k++)
    _sdio->sd_fifa = buf[k];
```

The second key difference is that the controller doesn't know when to write to the device or when to read from it. In general, the controller doesn't know the meanings of any of the commands. Therefore, if a command needs to use the FIFOs, the controller needs to be told. Likewise, if the data is supposed to flow from the FIFO to the device instead of from the device to the FIFO, the controller also needs to be told. For this reason, a separate bit needs to be set in addition to the previous ones—one to indicate that this is a write and not a read.

```
const    SDIO_WRITE    = 0x00000400,
          SDIO_ACK      = 0x04000000,
          SDIO_WRITEBLK = (SDIO_CMD | SDIO_R1 | SDIO_ERR
                           | SDIO_ACK | SDIO_WRITE | SDIO_MEM) + 24;

_sdio->sd_data = sector;           // Assumes a high density card
_sdio->sd_cmd   = SDIO_WRITEBLK;

wait_while_busy();
```

Also, by setting the SDIO_ACK bit, the controller will know to wait, for the acknowledgment token once it has completed its transmission. Failure to receive this token, whether because the NAK token was returned or no token was returned, will result in an error when set.

4.11 Writing multiple sectors

Writing multiple sectors to the device is very similar to writing a single sector. Indeed, the instructions for writing the first sector are identical, save that the command is now a CMD25 instead of a CMD24. Things become different when it comes to writing subsequent sectors, and then when it comes to ending transmission.

Let's walk through these changes one by one.

As before, we still have to read our data into the FIFO before we can issue the write command.

```
s = 0;

unsigned *src = &buf[s*512];

for(int w=0; w<512/sizeof(uint32_t); w++)
    _sdio->sd_fifa = src[w];
```

We then issue a command 25 to write multiple blocks to the device.

```
const    SDIO_FIFO    = 0x00001000;

_sdio->sd_data = sector;
_sdio->sd_cmd  = (SDIO_CMD | SDIO_R1 | SDIO_ERR
                 | SDIO_ACK | SDIO_WRITE | SDIO_MEM) + 25;
```

Then we come to our first change: we don't wait yet for the first block transfer to complete. Instead, we load the next sector into the second FIFO. In general, we'll alternate which FIFO we use.

```
for(s=1; s<sector_count; s++) {
    unsigned *src = &buf[s*512];

    if (s&1) {
        for(int w=0; w<512/sizeof(uint32_t); w++)
            _sdio->sd_fifb = src[w];
    } else {
        for(int w=0; w<512/sizeof(uint32_t); w++)
            _sdio->sd_fifa = src[w];
    }
}
```

Once the second sector has been transferred to FIFO memory, we can then wait for the current write transaction to complete and issue a new write request for the next sector.

```
while(_sdio->sd_cmd & SDIO_BUSY)
    ;

_sdio->sd_cmd = (SDIO_WRITE | SDIO_MEM)
               + ((s & 1) ? SDIO_FIFO : 0)
```

At this point, we repeat until we've written all of the sectors we wish to write to. Once we are done, we can send a command to `STOP_TRANSMISSION`, expecting an R1 response.

```
// First, wait for the last transmission to complete
while(_sdio->sd_cmd & SDIO_BUSY)
    ;

// Then send a STOP_TRANSMISSION request
_sdio->sd_data = (RCA << 16); // EMMC requires the RCA here
_sdio->sd_cmd = (SDIO_CMD | SDIO_R1 | SDIO_ERR) + 12;
```

4.12 Reading multiple sectors

Reading multiple sectors is almost identical to writing multiple sectors, save that the operation proceeds in reverse: we issue the read command first, and then copy data out of the FIFOs second.

First, the command is issued to read multiple sectors, given a starting sector.

```
_sdio->sd_data = sector;           // Assumes a high capacity card
_sdio->sd_cmd = (SDIO_CMD|SDIO_R1|SDIO_MEM) + 18;
```

We then wait for that command to complete.

```
while(_sdio->sd_cmd & SDIO_BUSY)
;
```

Once complete, we immediately switch FIFOs and issue a command to store the read data from the next sector into the next FIFO. This is also a good time to check for any read errors that may have taken place while reading the last sector.

```
for(s=1; !err && s<sector_count; s++) {
    while(_sdio->sd_cmd & SDIO_BUSY)
        ;

    if (0 != _sdio->sd_cmd & SDIO_ERR)
        err = 1; // No more commands to issue on error
    if (s + 1 < sector_count && !err) {
        // Immediately start the next request
        _sdio->sd_cmd = SDIO_MEM + ((s&1) ? 0:SDIO_FIFO) + 18;
    }
}
```

This second command must be issued before the device starts sending the second sector of data. One way to make sure of this is to set the clock shutdown bit in the PHY. That way, the clock won't toggle between when the first block is received and the second block request has been made.

Once the second read request is away, we have until it finishes to copy the data out of the first FIFO.

```
unsigned *dst = &buf[s*512];

if (s&1) {
    for(int w=0; w<512/sizeof(uint32_t); w++)
        dst[w] = _sdio->sd_fifa;
} else {
    for(int w=0; w<512/sizeof(uint32_t); w++)
        dst[w] = _sdio->sd_fifb;
}
```

These operations are time sensitive, and could create a race condition. Specifically, the host must request the next sector before the SD card starts to send it.

It is recommended to reduce this time pressure by setting the clock shutdown bit in the PHY. If this bit is set, the SDIO/eMMC clock will stop toggling when not in use. This should allow the software driver enough time to finish copying out any data, and to issue a second read command before the data starts showing up.¹

Once all transfers have completed, a stop transmission command needs to be sent for the card to stop sending more data.

¹There is an unwritten requirement here, however, that the card must provide at least a break of a couple of system clocks plus at least one SDIO clock in order for this clock stop to happen in time.

```

while(_sdio->sd_cmd & SDIO_BUSY)
    ;

// Send a STOP_TRANSMISSION request
_sdio->sd_data = (RCA << 16); // EMMC requires the RCA here
_sdio->sd_cmd = (SDIO_CMD | SDIO_R1 | SDIO_ERR) + 12;

```

The software driver can then copy the last sector out of the FIFO.

```

unsigned *dst = &buf[s*512];

if (s&1) {
    for(int w=0; w<512/sizeof(uint32_t); w++)
        dst[w] = _sdio->sd_fifa;
} else {
    for(int w=0; w<512/sizeof(uint32_t); w++)
        dst[w] = _sdio->sd_fifb;
}

```

Do note that we are reading the FIFOs in reverse. When `s==0`, we request FIFO zero. However, when we come back to read from that FIFO `s` will have advanced, so it will look like we are reading opposite of what we have requested even though this isn't the case.

4.13 Using the DMA to write multiple sectors

It's even easier to write multiple sectors using the DMA. In this case, you only need to make one request and then wait for it to complete.

```

_sdio->sd_dma_address = (char *)src;
_sdio->sd_dma_length = sector_count;
_sdio->sd_data = sector; // Assumes a high capacity card
_sdio->sd_cmd = (SDIO_CMD | SDIO_R1 | SDIO_ERR | SDIO_DMA
                | SDIO_WRITE | SDIO_MEM) + 25;

```

All that's required next is to wait for the operation to complete.

```

while(_sdio->sd_cmd & SDIO_BUSY)
    ;

```

In this case, all the ping-pong buffer handling will be handled internally by the DMA controller. Likewise, there's no need to send any `STOP_TRANSMISSION` command, since the DMA will send it automatically once all sector data has been transferred to the card.

Unlike using an external Wishbone DMA, there's no requirement to first read data from the controller before writing it back to the bus (i.e. to memory). Hence, the number of clock cycles required for a single sector of this write operation will be given by 512 divided by the bus width, times any throughput delays, plus the bus/memory latency. This will be nearly twice as fast as when using an external DMA, such as the ZipCPU's DMA.

4.14 Reading multiple sectors with the DMA

As with writing to the SD card using the DMA, reading from the card is just a matter of setting four configuration registers and then waiting.

```
const    SDIO_DMA    = 0x00002000;

_sdio->sd_dma_address = (char *)dst;
_sdio->sd_dma_length  = sector_count;
_sdio->sd_data        = sector;
_sdio->sd_cmd         = (SDIO_CMD | SDIO_R1 | SDIO_ERR | SDIO_DMA) + 18;
```

As with DMA writing, all that's required next is to wait for the operation to complete.

```
while(_sdio->sd_cmd & SDIO_BUSY)
    ;
```

Again, there's no need to send any `STOP_TRANSMISSION` command, since the DMA will send it automatically once all sector data has been transferred to the card.

Unlike the multisector read command above, there's no reason to adjust the clock stop bit of the PHY register. When using the DMA to read, the IO clock will be stopped automatically as necessary between commands.

4.15 Using the DMA streaming interface to write multiple sectors

If the design has been built with both `OPT_DMA` and `OPT_STREAM`, then data may be streamed to the device from an external AXI stream interface. This interface may be driven by an external DMA, or perhaps some (other) external data generation capability. Using the stream interface is nearly as simple as the DMA alone.

The first step, however, is external to the SDIO controller: the stream data generator must be set up to generate the requested data. How this is to be done is outside of the scope of this user guide. This configuration should be set to produce as much data as the command intends to consume. That is, all of the data should be produced at once, and fed to the controller via the AXI stream interface, throttled only by the stream interface's back pressure.

I'll note here that the SDIO controller has no insight into where external data packets begin and end. The `TLAST` packet delimiter is thrown away and so not used.

Once the external source is set up, the only difference in setting up the transfer is that the DMA address must have the `STREAM` bit set. The easiest way to do this is simply to issue a request to read from the negative one address.

```
char    *src = (char *)-1;
_sdio->sd_dma_address = (char *)src;
```

No other changes are required when all goes well.

However, the SDIO DMA controller will stop if and when it encounters an error. If this happens, the external stream source may still need to be flushed. Such a flush may violate the AXI stream

protocol, and will result in a loss of data. How to make this happen is outside of the scope of this user guide.

4.16 Reading multiple sectors to feed the DMA streaming interface

If the design has been built with both `OPT_DMA` and `OPT_STREAM`, then data may be streamed from the device to an external AXI stream interface. This interface may then drive an external DMA, or perhaps some (other) external consuming peripheral.

The setup of this external peripheral is a touch different, due to the presence of the `TLAST` signal coming from the design. This signal will be set on the last beat of every block in the transfer. As a result, the external DMA must either ignore this signal, be reconfigured for a multiblock transfer, or it must be reconfigured before every block in the transfer.

Once the external source is set up, the only difference in setting up the transfer from a normal DMA transfer is that the DMA address must have the `STREAM` bit set. As with stream writes, the easiest way to do this is simply to issue a request to write to the negative one address.

```
char    *src = (char *)-1;
_sdio->sd_dma_address = (char *)src;
```

No other changes are required.

4.17 GO IRQ STATE

The one command that the IP knows by number is the GO IRQ State command, `CMD40`. This is an eMMC only command. It needs to be treated special, because it has no timeout when waiting for a response. It also needs to be treated special, so that the controller can end the command if necessary.

The first step in the `GO_IRQ_STATE` command is to switch to an open-drain IO mode. This also means that the clock must be slowed down as well.

```
_sdio->sd_phy = (_sdio->sd_phy & ~(SDIO_PUSH_PULL | 0xff))
               | SDIOCK_5MHZ;
```

We can now issue the `GO_IRQ_STATE` command.

```
_sdio->sd_data = 0;
_sdio->sd_cmd = SDIO_READREG + 40;
```

At this point, we need to wait for either an SDIO/eMMC controller interrupt, indicating that the eMMC command has completed and an `R1` response has been read, or an operating system timeout external to the controller indicating something has gone wrong and we don't wish to wait any longer.

In the first case, if the chip returns an `R1` response, then the command is complete and we are done. We can clean up by restoring the PHY to its original mode. No other actions are required.

However, if the software device driver decides not to wait any longer, then we need to create a self-reply to this command.

```

_sdio->sd_data = 0;
// Note that the SDIO_CMD bit is must *not* be set for this
// command to interrupt the controllers natural wait.
_sdio->sd_cmd = 40;

```

Once this command completes, we are done and can return the PHY to its original mode. No valid data will be returned from this command.

There exists the possibility that this command will step on top of the actual return from the eMMC card. Electrically, this should be okay since we've returned to open drain mode. If the parameter `OPT_COLLISION` is set, then the front end will attempt to detect such a collision and automatically back off². If not, the collision may result in a command response with a bad CRC. Either way, the card will leave its IRQ state.

4.18 eMMC BOOT

The eMMC standard allows for two separate types of boot commands, a normal one and an alternate boot command. If `OPT_EMMC` and `OPT_BOOTEN` are both set, then the design will support both.

The first type of boot command may be entered following a hardware reset. No software interaction is required, save to wait for the boot to complete. In this case, the user will want to set the following parameters:

- Set `OPT_AUTOBOOT` to enable an automatic boot upon reset
- Set `BOOT_TOKEN` if the eMMC has been configured to acknowledge the boot request before sending any data (recommended)
- Set the `BOOT_ADDR` to the memory address where the BOOT information is to be stored by the DMA (assuming `OPT_DMA` is also set).³
- Set the number of blocks (512 Byte sectors) to be transferred as part of any BOOT operation via `BOOT_BLOCKS`
- Set the default clock speed for booting via `BOOT_SPEED`.
- Set the protocol used for booting via `BOOT_MODE`.
 - Set `BOOT_MODE[3]` to use the data strobe, DS, pin to control read sampling. (i.e. HS400)
 - Set `BOOT_MODE[2]` to BOOT in DDR mode.
 - The bottom two bits of the `BOOT_MODE` control the width, using the enumeration shown in Tbl. 5.9. Hence, `BOOT_MODE[1:0]==2'b10` will set an 8b boot width, `BOOT_MODE[1:0]==2'b01` setting a 4b transfer width, and `BOOT_MODE[1:0]==2'b00` setting a 1b transfer width. (Note that a 1b transfer width is not the same as SPI mode.)

²As of this writing, there are known issues with the `OPT_COLLISION` implementation. It is designed to guarantee the design can still “hear” a response from the card. As a result of these issues, that response may come back corrupted.

³While an automatic boot may take place without using the DMA, in many ways this defeats the purpose: why load boot information in this manner if the CPU already has boot instructions?

The startup sequence will then start with an external reset, assuming `OPT_HWRESET` is set. Once this reset has been set for a minimum number of cycles (1uS), the reset will clear and the command line will be held low. If `BOOT_TOKEN` is set, the design will wait for the device to acknowledge the boot request. Then it will wait for the device to send `BOOT_BLOCKS` blocks of data, copying each to RAM as it arrives. Once complete, an interrupt will be generated and the automatic boot cycle will be complete.

This cycle can also work without the DMA. In this case, the user will be required to unload each block from the FIFO as its data becomes available.

This same sequence may also be commanded by the CPU via the control port. Since the normal boot mode must be entered following a reset, the design should first be placed in reset. This may be done via the external reset pin,

```
_sdio->sd_cmd = EMMCHWRESET;
```

or via a special CMD0 (GO-IDLE) command.

```
_sdio->sd_data = 0xf0f0f0f0;  
_sdio->sd_cmd = 40;  
  
wait_while_busy();
```

Once the device has been reset, the design parameters may be entered and/or adjusted as necessary.

```
//      SDIO_ERR clears any prior errors  
//      The R1b response to a null-command invokes the boot  
const    SDIO_BOOT    = 0x04008300;  
  
// Query the device capabilities  
_sdio->sd_phy = SDIOCK_200MHZ | SDPHY_WTEST | SDPHY_DDR  
              | SDPHY_SHFTMSK | SPEED_CLKOFF | SECTOR_MAX;  
// ... as discussed above ...  
  
// Set the device for the appropriate boot mode  
_sdio->sd_phy = BOOT_PHY_SETUP; // This would need to be user defined...
```

If using the DMA, we can also set the DMA (destination) address, and the number of blocks to transfer.

```
// Set up the DMA for the boot  
_sdio->sd_dma_addr = mem_destination;  
_sdio->sd_dma_length = block_count;
```

Finally, we can issue the boot command and wait for the subsequent interrupt.

```
CLEAR_INT(SDIO_INT); // Clear any current interrupts  
  
// Command the boot  
//      SDIO_ACK tells the controller to expect a card acknowledgment  
_sdio->sd_cmd = SDIO_BOOT | SDIO_ACK;
```

```
// Now wait for the boot to complete  
WAIT_INT(SDIO_INT);      // Wait for the given interrupt
```

Note that if the hardware reset bit has been made active, IP will hold the hardware reset active until its release. This release command can be given at the same time as the BOOT command. Even if this release request comes before the full eMMC reset has been completed, the hardware will still complete the reset before starting the BOOT sequence.

We now switch to the alternative boot sequence. Unlike the normal boot sequence, the alternate boot sequence is closer to the normal flow of the IP described elsewhere in this document. First, the eMMC is reset via the special GO-IDLE command.

```
_sdio->sd_data = 0xf0f0f0f0;  
_sdio->sd_cmd = 40;  
  
wait_while_busy();
```

Before issuing the alternate boot command, it makes sense to setup the DMA.

```
// Set up the DMA for the boot  
_sdio->sd_dma_addr = mem_destination;  
_sdio->sd_dma_length = block_count;
```

Now we issue a second GO-IDLE command, this time with the argument required to activate the alternate boot mode.

```
const SDIO_ALTBOOT = 0x00008040;    // This is also a GO-IDLE CMD  
  
_sdio->sd_data = 0xffffffffa;  
_sdio->sd_cmd = SDIO_ALTBOOT | SDIO_ACK | SDIO_DMA;  
  
wait_while_busy();
```

Once the command completes, all that's left is to check the status, handle any potential errors, and boot the CPU.

5.

Registers

As mentioned in the last two chapters, the SDIO IP supports five registers, and three additional DMA-only registers. These are shown in Tbl. 5.1. Of these registers, the command register, CMD,

Name	Address	Width	Access	Description
CMD	0x00	32	R/W	SDSPI Command and status register
ARG	0x01	32	R/W	SDSPI return data/argument register
FIFO[0]	0x02	32	R/W	FIFO[0] data
FIFO[1]	0x03	32	R/W	FIFO[1] data
PHY	0x04	32	R/W	Front end configuration register
DMA Addr	0x05	32	R/W	DMA address bits[31:0]
DMA Addr	0x06	32	R/W	DMA address bits[63:32]
DMA LEN	0x07	32	R/W	DMA transfer length in blocks
PHY TRIM	0x08	32	R/W	Control sub-clock input delays
RX TRIM	0x09	32	R/W	Controls RX clock pre-shutdown

Table 5.1: (Big-Endian) I/O Peripheral Registers

initiates and provides feedback from all commands to/from the external device, so we'll spend most of our time discussing it.

5.1 CMD Register

Only writes to the CMD register will initiate interactions with the external device. As a result, it is central to how the entire IP operates. The CMD register itself is composed of several packed bit fields, as shown in Fig. 5.1. The various fields of this register are explained in Tbls. 5.2 and 5.3.

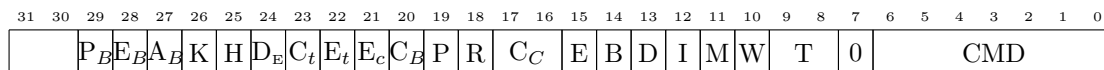


Figure 5.1: CMD Register fields

Name	Width	Description
P_B	1	OPT_BOOTEN, can be used to tell if the design has been built with the BOOT capability enabled.
E_B	1	BOOT Error. This will be set if any error, such as a failure to return a boot acknowledgment or a DMA error, takes place during BOOT activity.
A_B	1	BOOT Active. This bit will be set as long as any BOOT operations are active.
K	1	Expect CRC token. Both eMMC and SDIO protocols return CRC acknowledgments (or negative acknowledgments) following the transmission of data sectors. eMMC also returns an acknowledgment following a BOOT request. If set, the controller will look for and expect such a token. If the token is not present within a minimal timeout period, an error will be declared. If not set, no such token timeout errors will be generated.
H	1	Hardware reset. Set to one to assert (lower) the <code>o_hwreset_n</code> wire. Internal logic will insist on a minimum $1\mu s$ (i.e. 100 clock) reset duration. This feature is only enabled if OPT_HWRESET is set.
D_E	1	DMA Bus Error. True if the DMA encounters a bus error while transferring data. This bit will also be set if the DMA address wraps at any time during the operation.
C_t	1	Data transfer error code. Following a transfer error, this bit will indicate the cause of the error. If the bit is zero, the design either timed out waiting for data or the transfer acknowledgment to arrive (assuming K was set). This bit will be set following either a receive CRC error, or a transmit negative acknowledgment token.
E_t	1	Transfer error. Set if the E bit is set due to a data reception error. This will also be set following a write where an ACK token is expected (via the K bit) and not received.
E_c	1	Command error. Set if the E bit is set due to an error in command wire processing.
C_B	1	Card is busy following an R1b command. This bit will be set if the card asserts a busy signal by lowering DAT[0] following an R1b command. It will clear when the busy clears.
P	1	No card present. Set if the card detect bit indicates no card is currently present. Automatically clear upon insertion. This facility is only available if the OPT_CARD_DETECT parameter is set.
R	1	Card removed. This bit will be set if both the OPT_CARD_DETECT parameter has been set, and the card detected bit has dropped since a card was last acknowledged. This bit will only be cleared by the user writing a one to this bit to clear it. As a result, it can be used to detect if a card has been inserted since the last setup, and thus needs to be set up.
www.opencores.com		Rev. 2.0pre

Table 5.2: CMD Register field descriptions, part one

Name	Width	Description
C_C	2	Command error code. Indicates the type of error, if any, received from the last command. This is separate from any potential bus or receive errors.
E	1	Error. Set if the last command or data transaction completed with error. Once set, further commands will be ignored until this bit is cleared. Write a one to this bit to clear it. Errors may be cleared concurrent with new transactions requests.
B	1	Busy. Either a command or data transfer is currently in progress.
D	1	DMA Enable. Set this bit to enable a DMA operation as part of an issued command. This bit is ignored if <code>OPT_DMA</code> is not set. This bit will be cleared once the DMA operation is complete.
I	1	FIFO selection. Keep clear to use FIFO A. Set to use FIFO B. When using the DMA, the DMA will begin with the selected FIFO, but then alternate between FIFOs on a block by block basis.
M	1	Memory operation. If set, the requested command also includes a data transfer to or from the FIFOs. If bit[6] is also set, this transfer will take place once the associated command is complete. Otherwise, if bit[6] is clear, the memory operation will take place without an associated prior command and response. This is useful for supporting multi-block transfers. This bit is self clearing once the operation is complete.
W	1	Write operation. If this bit is set for a memory operation, then data will be transferred from a FIFO to the device. Otherwise, any memory operation is a read operation, which will transfer data from the device to the FIFO. This bit is self clearing once the write operation completes.
T	2	Response type. Indicates whether a response is expected, and whether or not that response will have 48'bits or 136'bits. See Tbl. 5.5 for more info.
CMD	8	The command. Writes with bits [7:6] set to 2'b01 will forward the other six bits of the command to the external device. Writes with bit [7] set can be used to communicate with the controller without sending an associated command to the device. Writes with both bits clear and the memory request bit set can be used to make requests of the memory channel without also requesting a preceding command transfer. Once any requested command transfer completes, these 8-bits will be replaced by the first 8-bits of the reply. Most such replies will typically just clear bit [6].

Table 5.3: CMD Register field descriptions, part two

Starting at the top, the first three bits are associated with the BOOT capability. P_B will be set if the design has been built with a BOOT capability within it.¹ E_B will be set following any BOOT operation error. If set, the error bit (bit 15) will also be set. Finally, A_B will indicate if a boot operation is currently active and ongoing.

Bit H is the reset bit. Anytime this bit is set, the reset pin output will be active (low). If set, the hardware reset will be set for a minimum of $100\mu s$. This pin must be cleared manually, allowing the external reset pin to be controlled by the CPU. While in reset, any command requests will be ignored.²

Because of this delay in clearing, the reset pin should be checked before going forwards.

```
// Activate the reset
_sdio->sd_cmd = SDIO.HWRESET | SDIO.NULLCMD;
if (_sdio->sd_cmd & SDIO.HWRESET) {
    // Hardware resets are supported.

    // Can immediately clear the reset
    _sdio->sd_cmd = SDIO.NULLCMD;
    // Now, wait for the reset to actually clear
    while(_sdio->sd_cmd & SDIO.HWRESET)
        ;
}
```

Commands are issued to the device by writing to this register with bits [7:6] set to 2'b01 and the reset bit is clear. There are some exceptions to this rule:

- If the error bit is set, then any command that doesn't also set bit 15 (clear error) will be ignored.
- If the device is already busy with another command, then any new command will be ignored.
- If the FIFOs are busy with a previous command, then any new command that would use the FIFOs will also be ignored.
- If the DMA is busy, then all new commands will be ignored. (Controller reset commands will still abort ongoing DMA operations.)
- If bits [7:6] are both zero, a new command will be issued if the device was configured with eMMC support (OPT_EMMC=1), no FIFO activity is requested, and no response is expected. This is to support the 'self-reply' capability designed to force a slave to exit the IRQ state. Also, if bits [9:8] are both set, then a BOOT sequence will be initiated.

While most writes to the command register will initiate commands or data interactions with the external device, there are several exceptions to this rule as shown in Tbl. 5.4. Other than any reset requests (hard or soft), GO_IDLE and self-replies to an eMMC GO_IRQ_STATE command, all writes to the command register will be ignored while it is busy.

Commands may expect one of four types of responses, as shown in Tbl. 5.5.

¹Note that this capability may be removed in a future release if it isn't found to be useful.

²The BOOT command is an exception to this rule. The BOOT command may be issued while the hardware is still in reset, as long as the BOOT command also releases the reset signal.

D	M	R2	[7:6]	Description
0	0	0	2'b00	If <code>OPT_EMMC</code> is set, a command response will be sent. This is useful for ending an IRQ waiting period if the IRQ hasn't taken place.
1	?	0	2'b00	No command will be sent, but a DMA operation will begin. If the reply selected is for an R1b response, this will initiate a BOOT sequence with DMA support.
0	1	0	2'b00	No command will be sent, but a memory operation will begin. If the reply selected is for an R1b response, this will initiate a BOOT sequence with FIFO instead of DMA support.
?	?	?	2'b01	A command will be sent to the card.
1	?	0	2'b01	The command will be followed by a DMA transaction.
0	1	0	2'b01	The command will be followed by a memory transaction using the FIFOs.
0	0	1	2'b01	The command response will be placed into the selected FIFO.
?	?	?	2'b1x	No commands will issue, nor DMA nor memory operations commence. Writes of this type may be used to reset the user FIFO position, to clear any error flags, or card removed bit(s)
32'h5200_0000				Soft reset request. This resets the controller internally. Any ongoing operations will be aborted. Unlike the hardware reset bit, no indication of this internal reset will be forwarded to the external card/chip.

Table 5.4: Types of writes to the CMD register

Resp Type	Description
2'b00	No response expected
2'b01	R1 (48b) response expected
2'b10	R2 (136b) response expected. Returned data will be placed into the selected FIFO.
2'b11	R1b (48b+BUSY) response expected. If bits[7:6] are also zero, this response type will instead initiate a reset.

Table 5.5: CMD response types

CMD Err Code	Description
2'b00	Timeout. The card has not responded. This will also be the return code for commands not requiring any response.
2'b01	Okay. No error was encountered, and the card returned a valid response.
2'b10	CRC Error. The response returned by the card did not have a valid CRC. Not all commands will have valid CRCs. Check the SDIO and eMMC specifications to know when to expect one.
2'b11	Frame error. Set if the CRC passes, but the stop bit of any command reply is not also set.

Table 5.6: CMD response types

If the memory bit is set, then the data FIFOs will be used in the operation. Which FIFO is used will be controlled by the FIFO ID bit. “Write” transactions, those that transfer data from the FIFO to the card, are identified by also setting the ‘W’ bit, otherwise all FIFO transactions will be read transactions by default.

If set, the “D” bit will enable a DMA operation. If the design has not been built with any DMA installed (`OPT_DMA=0`), an error will result instead.

As soon as a command has been requested, the busy bit will be set. It will remain set while the command is being sent. If the command expects a response, then it will remain set while the controller awaits the response. If the command also expects a data transfer, then the busy bit will remain set while the data transfer takes place. Once the entire operation, command, reply, and optional data transfer, have completed then the busy bit will be cleared.

The controller will return one of four results for any reply request, as shown in Tbl. 5.6. In general, any non-zero response indicates that a start bit was received.

If the `OPT_CARD_DETECT` parameter is set, then the controller will be responsive to a “card detect” input. Whether or not a card is currently present can be determined by the “No card present” bit. If ever a card is removed from the system, this bit will be set. A second bit, the card removed bit, will also be set if no card is present. Once a card is inserted, the “No card present” bit will clear automatically leaving only the “Card removed” bit to be cleared manually. This helps to insure that software can always detect any time a card has been removed, since the “Card removed” bit will be set until cleared.

If the `OPT_CARD_DETECT` parameter is clear, then the “No card present” bit as well as the “Card removed” bit will always read zero.

If, following any R1b command, the card holds the DAT[0] line low, this will be reflected in the “Card busy” bit. Once the card raises DAT[0], the “Card busy” line will be cleared automatically and will not reassert until the next R1b command.

If, following any command, the card holds the DAT[0] line low, this will also be reflected in the “Card busy” bit.

The error bit may be set from either a command or a data transfer error. The DMA and BOOT controllers may also set the error bit. The E_B , D_E , E_t and E_c bits exist so you can know which of these types of errors triggered the error bit.

Data transfer error codes are listed in Tbl. 5.7.

Rx Err Code	Description
1'b0	Watchdog timeout. When receiving, this bit will be set if no bits are received within the watchdog timeout period. This bit will also be set if the clock (or data strobe) stop mid-reception. If <code>OPT_CRCTOKEN</code> and the <i>K</i> bit are set, then this bit may also be set following a data block write transfer if neither acknowledgment nor negative acknowledgment tokens are received.
1'b1	CRC error. When receiving, this bit will be set if a block (sector) is received with an invalid CRC. If <code>OPT_CRCTOKEN</code> and the <i>K</i> bit are set, then this bit may also be set following a data block write transfer if the negative acknowledgment token is received.

Table 5.7: Potential receive error codes

5.2 Argument register

According to both SDIO and eMMC specifications, all commands sent to the device have a 48-bit format. The first 8-bits specify a start bit followed by the 7-bit command itself, where commands from the host begin with a 2'b01 (including the start bit). The next 32-bits constitute an argument to the command. This argument register controls those next 32-bits.

Writes to this register, prior to issuing a command, will set the argument for the next command.

Following any 48-bit reply from the device, the 32-bit argument from that reply may be read from this register.

5.3 FIFO Registers

As with the SDSPI controller, the SDIO/eMMC controller maintains two (nominally) 512 byte memory areas called FIFOs.³ Reads from the card will write data into one of the two FIFO's, whereas writes to the card will read data out from one of the FIFO's. Which FIFO the card uses is determined by the *I* bit in the CMD register (See Fig. 5.1).

Further, upon any write to the CMD register, the FIFO address will be set to point to the beginning of the FIFO.

The purpose of the FIFOs is to allow one to issue a command to read into one FIFO, and then when that command is complete to read into a second FIFO. While the second command is ongoing, a CPU or DMA may read the data out of the first FIFO and place it wherever into memory. Then, when the second read is complete, a third read may be issued into the first buffer while the data is read out of the second and so forth.

This interleaving approach, sometimes called ping-pong buffering, can also be used for writing: A multi-sector write might start by first writing into one FIFO. The write command would then be issued, followed by writing into the second (now unused) FIFO. Once the first write completes,

³Actual FIFO size is determined by the `LGFIPO` parameter.

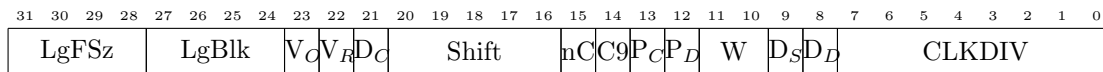


Figure 5.2: PHY Front end configuration register

a second write command may be issued while the first FIFO is refilled for a third command and so forth. Feel free to examine the various included software drivers for examples of how this may be accomplished.

One item to note before closing: there is only one internal address register when accessing the FIFO from the Wishbone bus (AXI-Lite) control port. Attempts to read from or write to either FIFO from the bus will increment this address register. Interleaved read, or write attempts, such as reading one item from FIFO[0] and writing another item to FIFO[1], will each increment the internal address pointer so that the result is likely to be undesirable. For this reason, it is recommended that only one FIFO be read from or written by the bus at a time.

The DMA also uses these FIFO registers, with the difference being that the DMA can fill or empty the FIFO's much faster than typical bus accesses can.

The FIFOs are never cleared by commands. They are only overwritten with data. Hence any attempt to read from the device that fails will likely leave the previous FIFO data in place. If necessary, the FIFO's can be cleared by software for security purposes.⁴

5.4 PHY Register

The PHY register controls the SPI clock rate, the sector size, and any other PHY IO options, such as the width of the bus or whether command and data wires are in push-pull or open drain modes. Specifically, with regards to the FIFO size, the PHY register controls how many bytes will be written into the FIFO (which is really of a fixed size) before expecting a CRC when receiving data, or equivalently how many bytes to read out of the FIFO before adding a CRC to any transmission. The fields of this register are shown in Fig. 5.2. and explained in Tbl. 5.8. Note that while every effort is made to comply with a user request, writes to the PHY register will also map illegal user requests to their closest supported capability.

The various width enumerations are shown in Tbl. 5.9. Width requests will be mapped to the nearest value the controller supports. Hence, a request for a 8b data width on a device only configured for 4 bits will quietly be downgraded to a 4 bit request. This feature may be used to find out how many bits the controller actually supports, as discussed in Sec. 4.3.

The CKDIV field needs the most explanation. This field is used to control a digital clock divider. Particular high speed frequencies do not go through the divider. All other frequencies may be expressed as divisions of 50 MHz, as shown in Tbl. 5.10⁵. Attempts to use the high frequency values, but without high frequency support, will be quietly mapped to the closest supported frequency. Indeed, this can be one way of telling whether or not controller support exists for either `OPT_SERDES` or `OPT_DDR`: set the clock to its highest speed, and see which speed is permitted, as in Sec. 4.3.

⁴There should be an example of how to clear the FIFOs found in the example driver software.

⁵This table assumes a 100 MHz system clock. For other system clock frequencies, these outgoing clock frequencies will scale appropriately.

Name	Width	Description
LgFSz	4	Reflects the LGFIFO parameter, and hence the log based two of the maximum transfer block size in bytes.
LgBlk	4	Log based two of the transfer block size. Set to 9 for 512 Byte transfers, 4 for 16'Byte transfers, etc. Cannot be set below 2.
V _O	1	Read only value reflecting the parameter OPT_1P8V. If set, indicates that the controller is capable of driving an output value to direct hardware to switch to a lower voltage.
V _R	1	1.8 Volt request & status. Set to transition to 1.8 Volts, or leave clear to operate at 3.3 Volts. On read, returns the current value from the top level. This value will only change if both OPT_1P8V is set, and external hardware is present to allow voltage changes. Once set, this register cannot be cleared without either a hardware reset or the card assertion pin (if supported) being withdrawn.
D _C	1	Set for enhanced data strobe support. If set, both the command and data pins will use the data strobe as a clock pin. Will only be set if D _S and D _D are also set.
Shift	5	Sample shift. This is used to determine IO sampling. The digital clock output signal will be delayed by one clock plus this many eighth clock units to determine the receive sample point when not using the data strobe. This value is likely to need to be adjusted from its simulation/demonstration settings when operating at high speeds in order to reflect IO delays contained in either the front end IO drivers, the external device, or delays across the circuit board itself.
nC	1	Clock shutdown. Set in order to deactivate the clock between commands, or leave clear for a continuously operating clock.
C9	1	Clock-90. Set in order to offset the clock from the data by ninety degrees, or leave clear for the clock to be offset from the data by 180 degrees. Any attempt to use DDR data transfer will automatically set this bit.
P _C	1	Push-Pull CMD. Set in order to drive the CMD wire via a push-pull scheme, or leave clear to use open-drain for CMD transfers.
P _D	1	Push-Pull Data. Set to drive data wires via a push-pull scheme, or leave clear to use open-drain for data transfer.
W	2	Width. Controls how many data bits are used in any data transaction. Set to 0 to use DAT[0] only, 2'b01 to use DAT[3:0], and 2'b10 to use DAT[7:0]. See also Tbl. 5.9. Defaults to BOOT_MODE[1:0] if OPT_AUTOBOOT is set, or 2'b00 otherwise.
D _S	1	Data strobe. Set to one to use the eMMC data strobe, clear otherwise. Requires DDR mode set, so D _D must be high.
www.opencores.com		Parameters OPT_EMMC and OPT_DS must also be set to use this capability. Defaults to BOOT_MODE[3] if OPT_AUTOBOOT is set, or 1'b0 otherwise.
D _D	1	DDR. Set to one to transmit on both edges of the clock. Clear to send data on the positive edge of the clock only. Defaults to BOOT_MODE[2] if OPT_AUTOBOOT is set, or 1'b0 otherwise.
CKDIV	8	Clock division control. See Tbl. 5.10 for more information. Defaults to BOOT_SPEED if OPT_AUTOBOOT is set, DEF_SPEED otherwise.

Table 5.8: PHY Front end configuration register fields

Width	Description
2'b00	1b data width. Only DAT[0] will be used to transfer data.
2'b01	4b data width. DAT[3:0] will be used to transfer data.
2'b10	8b data width. DAT[7:0] will be used to transfer data. Only used in eMMC protocols.
2'b11	Reserved/unused.

Table 5.9: Width enumeration

CKDIV	Clock Frequency
8'h0	200 MHz clock. Requires OPT_SERDES.
8'h1	100 MHz clock. Requires either OPT_SERDES or OPT_DDR.
8'h2	50 MHz clock
8'h3	25 MHz clock
8'h4	12.5 MHz clock
N	Sets clock frequency to $25/(N - 2)$ MHz.

Table 5.10: Clock division settings

This controller supports both SDR and DDR IO modes. DDR IO, however, requires that the clock be offset from the data by 90 degrees rather than the nominal 180 degrees.⁶ While the `OPT_SERDES` driver will support 90 degree clock and data offsets at all frequencies, the `OPT_DDR` driver will only do so at 50 MHz and below. Any attempt to enable DDR mode will automatically place the clock in its 90 degree offset mode. Alternatively, the 90 degree offset mode may be enabled specifically—even if not required.

The eMMC controller contains the option of using a data strobe to clock the return data. This requires both `OPT_EMMC` and `OPT_DS`. This mode may be enabled in the PHY register. Doing so will force all data bits returned by the external device to be clocked using this return data strobe. A separate control bit, D_C enables “enhanced data strobe support” as well. If set, command reply bits will also be clocked by the return strobe.

The SDIO/eMMC IO clock may be turned off when not in use. To turn this clock off, set the clock shutdown bit in the PHY register. Note that the DMA may automatically shutdown the clock when reading as well.

The shift field deserves some attention. This field is used to determine when to sample returned data, with respect to an outgoing clock. It needs to be set to the time required both for the data to be sent to the IO pins, plus any round trip time across the PCB and through the device, to the return time from incoming pin to bit detection. For slower speeds, a shift of zero will typically suffice. For faster speeds, getting the shift right will take some work. Failing to get the shift right will likely result in receive CRC failures.

Finally, if `OPT_1P8V` is set, then the design contains a means within it of adjusting voltages. This capability is designed to control a general purpose IO bit, `o_1p8v`, which when/if set is designed to

⁶DDR support may also require 1.8V IO. Check the SDIO/EMMC specifications and your board support for more information.

control an external voltage translator. The `o_1p8v` value is clear upon reset, so that the external voltage can be set to 3.3 V. It may later be set, causing the external voltage controller to switch to 1.8 V. Consistent with the eMMC specification, there is no way to unset this value. This pin is typically incompatible with most FPGAs, since FPGA IO voltages are fixed in any given configuration.

Even if voltage switching is not supported, the V_R bit may be used to tell the design which voltage bank it is currently operating in. If clear, software will know that the IO pins are operating in 3.3 Volt mode. If set, the IO pins are operating in 1.8 Volt mode. This can be used to help determine which speeds are available to the controller.

The `LgBlk` field sets the log, base two, of the any memory transfer size. The actual transfer length will be 2^{LgBlk} bytes. The WB controller will not allow this to be set larger than the size of the FIFO, as reported by the `LgFSz` field, so any attempt to set this value past its maximum can be used to read back what the maximum value actually is.

5.5 Soft Resets

The controller may be reset at any time by writing a `32'h52000000` to the CMD register.

Any ongoing transactions will be canceled at that time and the controller will return immediately to idle. Note that this reset *does not* send any commands to the card, it simply resets the controller. Any ongoing operations will be halted mid-operation. If the card is in the middle of a multi-block operation, an additional stop transmission command may be required to bring the card back into a usable state.

Alternatively, a `GO_IDLE` command may be sent at any time—even if a command response is pending or an IO operation is ongoing.

If the hardware reset pin is supported via `OPT_HWRESET`, then any write setting the H bit (bit 25) of the command register (and so clearing the `o_hwreset_n` pin) will also reset the controller. Unlike the soft reset request described above, this reset will be forwarded to the external device via the hardware reset pin. The specific soft reset sequence has been carefully designed so as to generate a hard reset if the hard reset pin is available.

While SDIO devices don't typically have hard reset pins, the `OPT_HWRESET` pin may still be used to power off the SDIO device—such as when it is necessary to change voltages back to 3.3 V.

5.6 DMA Usage

An optional DMA may be integrated into this controller. Additional registers are required to use the DMA. These include the DMA length register, and the DMA address register. These registers are shown in Tbl. 5.11. To use the DMA, these two additional registers must first be set before

Offset	Little Endian	Big Endian
0x05	DMA Address bits[31:0]	DMA Address bits[63:32]
0x06	DMA Address bits[63:32]	DMA Address bits[31:0]
0x07	DMA transfer length in blocks	

Table 5.11: (Big-Endian) I/O Peripheral Registers

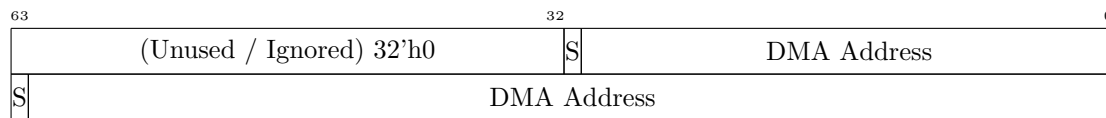


Figure 5.3: DMA Stream config bit location for < 31b or greater address widths

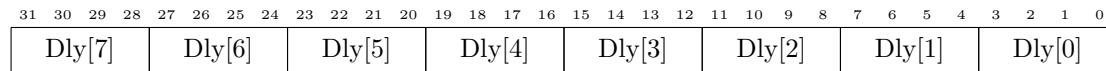


Figure 5.4: PHY Front end configuration register

issuing the command requesting that the DMA be used.

The DMA length register controls the number of *blocks* the DMA will transfer. The block length can be set in the PHY register, and it is required to be a power of two. Once the DMA has transferred this many blocks (from alternating FIFOs), the DMA operation will end.

The DMA address register controls where on the bus data will be drawn from or written to. During a write operation, data will be read from the bus and written to the card. During a read operation, data will be read from the card and written to the bus. The direction of the operation is controlled by the command word when the DMA operation is requested. Further, during any DMA operation, this register will increment as data is written to or read from the card.

The DMA address register is designed to be a 64 bit register, to support potentially large address spaces. This can be an issue with smaller address spaces. The first issue, as shown in Tbl. 5.11, is that the lower 32 bits of the address register will move according to whether or not the design is built for big or little endian addressing.

The second issue with smaller address spaces is associated with stream addressing, as shown in Tbl. 5.3. If `OPT_STREAM` is set and the address width is less than 31 bits, then the streaming address bit will be bit 31. That is, if the stream address bit fits in 32 bits, it will be placed in the MSB of the least significant 32 bits. Otherwise the streaming address bit is bit 63. In both cases, the stream address can be set by setting the DMA address to negative one.

Once the DMA registers have been set, all that remains is to set the DMA bit in the CMD register when requesting a command. A typical read DMA command will set the command bit, request an R1 response, and request a multiblock read. A typical write DMA operation will look identical save that the write bit will be set and the command will indicate a multiblock write request of the card.

DMA operations during BOOT mode will be terminated by either raising the CMD line (normal BOOT mode), or by sending a `CMD0 (GO-IDLE)` command with a zero argument. Otherwise, all DMA operations will be terminated with a `CMD12 (STOP_TRANSMISSION)`.

5.7 PHY TRIM

When sampling via the DS source clock, it may be a challenge to get timing done properly. For this reason, the PHY TRIM register exists—to allow the timing associated with each. As shown in Fig. 5.4, and Tbl. 5.8, each IO bit is allowed a delay from 0–15.

Name	Width	Description
Dly[x]	4	Controls a per-IO bit sub-clock delay, for the purpose of adjusting sampling timing.

Table 5.12: PHY Front end configuration register fields

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
(Reserved)												Rx Trim				DS Dly				CMD Dly											

Figure 5.5: PHY Front end configuration register

In the case of a Xilinx FPGA, the delay values are used to control the bottom 5 bits of an IDELAYE2 module.

The default (nominal) value of 0 should be good enough for most purposes.

5.8 RX TRIM

The receive trim register is designed to allow a configuration to exist which will shut down the clock prior to all data being received. The goal of this approach is to help stop the clock on a dime once all data has been requested, and once the rest of any incoming data remains in the pipeline. Apart from this capability, it is possible for an external device to begin sending a next block of data as part of a multi-block read operation before the controller is ready for it. This is a particular issue at higher speeds, such as HS400. If this happens, the controller will likely miss the start bit of that second block and (assuming it's not all ones) then fail its CRC. To prevent this from happening, the clock must stop long enough between blocks to allow the controller to reset itself prior to receiving the next block. This control helps to make sure the clock stops early enough at higher speeds.

The PHY TRIM register controls the incoming delays across all of the data pins associated with the design. The RX TRIM register finishes the optional delays, by allowing for a user controlled pre-sample delay for both the CMD and DS pins.

Fig. 5.5, and Tbl. 5.8, show the allocation of the bits for both the RX TRIM and the two remaining incoming pin delay controls.

Name	Width	Description
Rx TRIM	4	Once the controller is this many clock cycles prior to receiving the last data, a clock shut down will be enabled (assuming it is also enabled in the PHY register).
DS Dly	4	Controls an incoming delay on the DS.
CMD Dly	4	Controls an incoming, pre-sample delay on the CMD wire.

Table 5.13: PHY Front end configuration register fields

The command pin's delay should nominally be identical to the IO pin delays from the TRIM register.

The DS pin's delay is different. This should nominally be set to achieve a 90 degree phase shift in the clock—so that the DS clock would sample incoming data in the center of the eye.

Turning to the RX TRIM register, with zero RX TRIM, the receive clock will stop once the last incoming clock's worth of data is registered. This includes both data and CRC. Hence, in 1b mode, a 512 Byte sector will require 4112 clocks containing return data, whereas in 8b mode the same sector would require 528 clocks. Once this number, less the RX TRIM, has been received, a flag will be set to allow the clock to stop. From that point on, no more clocks will be generated for this receive block.

In the case of HS400, the number of 100 MHz system clock cycles of return data per block would be 132 (at a 200 MHz clock). Adjusting the receive trim register will shut the clock down before this number has been received, by however much the trim value is set to.

The “correct” value for the RX TRIM is a matter of active research. If this delay is too large, the controller will likely hang on every block. (A soft reset should be able to resolve this, to allow proper testing.) If this delay is too small, it is possible for the external device to start sending a subsequent block in a multi-block transfer prior to the controller being ready. It is quite likely that this “correct” value will be clock rate dependent.

6.

Interrupts

An interrupt may be generated for any of the following reasons:

1. Any time a command completes, and any anticipated response is either returned or timed out. The exception to this rule is that no interrupt will be generated if either the DMA is active, or if subsequent IO to or from FIFOs is still pending and the command request completes without error.
2. If `OPT_DMA` is set, and the DMA is included, then an interrupt will be generated once a DMA operation is completed. During the DMA operation, however, all other interrupts will be suppressed.
3. An interrupt will be generated in non-DMA mode any time a block (sector) is finished being sent to the controller.
4. An interrupt will be generated in non-DMA mode any time a block (sector) finishes being received by the controller.
5. If both `OPT_CRCTOKEN` and `OPT_BOOTEN` are set, and a `BOOT` command is given that does not use the DMA, then an interrupt will be generated with the `ACK` or `NAK` is received from the external device.
6. During any `BOOT` operation that does not use the DMA, an interrupt will be generated following the transfer of any block.
7. An interrupt will also be generated at the end of any `BOOT` operation.
8. If the `OPT_CARD_DETECT` parameter is set, then an interrupt will be generated any time an acknowledged card is removed.
9. If the `OPT_CARD_DETECT` parameter is set, then an interrupt will also be generated any time a card is inserted. Unlike the other interrupts, this interrupt will only clear once the “Card removed” bit is also cleared.

All but the last of these reasons will generate a single interrupt pulse, one clock wide.

7.

Wishbone Datasheet

Tbl. 7.1 describes the Wishbone command bus interface. It is required by the Wishbone specification,

Description	Specification	
Revision level of Wishbone	WB B4 spec	
Type of interface	Slave, (Block/pipelined) Read/Write	
Port size	32-bit	
Port granularity	8-bit	
Maximum Operand Size	32-bit	
Data transfer ordering	(Optional)	
Clock constraints	(See below)	
Signal Names	Signal Name	Wishbone Equivalent
	i_clk	CLK_I
	i_wb_cyc	CYC_I
	i_wb_stb	STB_I
	i_wb_we	WE_I
	i_wb_addr[3:0]	ADR_I
	i_wb_data	DAT_I
	i_wb_sel	SEL_I
	o_wb_ack	ACK_O
	o_wb_stall	STALL_O
	o_wb_data	DAT_O

Table 7.1: Wishbone Slave Datasheet

and so it is included here. Note that all Wishbone operations may be pipelined, to include FIFO operations, for speed.

The particular constraint on the clock, shown in Chapt. 8, is not really a Wishbone (or AXI-Lite) constraint, but rather an SD-Card constraint. Not all cards can handle clocks faster than 25 MHz. For this reason, the Wishbone clock, which forms the master clock for this entire controller, must be divided down so that the external SDIO device clock is within the limits the card can handle.

When the `OPT_DMA` parameter is also set, a Wishbone master interface will also be enabled, as outlined in Tbl. 7.2. Tbl. 7.2

Description	Specification																						
Revision level of Wishbone	WB B4 spec																						
Type of interface	Master, (Block/pipelined) Read/Write																						
Port size	32-bit or larger																						
Port granularity	8-bit																						
Maximum Operand Size	Full bus width																						
Data transfer ordering	(Optional)																						
Clock constraints	(Same)																						
Signal Names	<table> <tr> <th>Signal Name</th><th>Wishbone Equivalent</th></tr> <tr> <td>o_clk</td><td>CLK_0</td></tr> <tr> <td>o_dma_cyc</td><td>CYC_0</td></tr> <tr> <td>o_dma_stb</td><td>STB_0</td></tr> <tr> <td>o_dma_we</td><td>WE_0</td></tr> <tr> <td>o_dma_addr</td><td>ADR_0</td></tr> <tr> <td>o_dma_data</td><td>DAT_0</td></tr> <tr> <td>o_dma_sel</td><td>SEL_0</td></tr> <tr> <td>i_dma_ack</td><td>ACK_I</td></tr> <tr> <td>i_dma_stall</td><td>STALL_I</td></tr> <tr> <td>i_dma_data</td><td>DAT_I</td></tr> </table>	Signal Name	Wishbone Equivalent	o_clk	CLK_0	o_dma_cyc	CYC_0	o_dma_stb	STB_0	o_dma_we	WE_0	o_dma_addr	ADR_0	o_dma_data	DAT_0	o_dma_sel	SEL_0	i_dma_ack	ACK_I	i_dma_stall	STALL_I	i_dma_data	DAT_I
Signal Name	Wishbone Equivalent																						
o_clk	CLK_0																						
o_dma_cyc	CYC_0																						
o_dma_stb	STB_0																						
o_dma_we	WE_0																						
o_dma_addr	ADR_0																						
o_dma_data	DAT_0																						
o_dma_sel	SEL_0																						
i_dma_ack	ACK_I																						
i_dma_stall	STALL_I																						
i_dma_data	DAT_I																						

Table 7.2: DMA Wishbone Master Datasheet

8.

Clocks

All of the clocks within this system are based upon the speed of the system clock, `i_clk`. This clock determines all other clock speeds within the design.

Name	Source	Rates (MHz)		Description
		Max	Min	
<code>i_clk</code>	Input	104	96	System clock. Nominally at 100 MHz.
<code>i_hclk</code>	Input			I/OSERDES clock. Must be 4x the <code>i_clk</code> rate, and must come from the same PLL or MMCM as <code>i_clk</code> . This clock input is only used if <code>OPT_SERDES</code> is set to cause the design to use the 8x SERDES based front end that requires it.
<code>o_ck</code>	Output	200	0.1	SDIO/eMMC interface clock. This signal is output over the interface at a frequency given by the clock divider controller. May also be turned off between commands.

The `i_clk` speed has been nominally fixed at 100 MHz. Based upon this speed, the controller can generate 200 MHz, 100 MHz, and 50 MHz IO clocks. Other clocks may be generated by integer divisions of 50 MHz.

Other system clock rates may also be possible. However, the design operates primarily within a single clock domain. If the system clock rate is changed, all other clock rates produced by integer divisions of the system clock rate will scale accordingly.

9.

I/O Ports

Table 9.1 lists the various ports used by this controller, excluding the DMA AXI stream ports.

Port	Width	Direction	Description
i_clk	1	Input	System Clock, nominally 100 MHz
i_hsclk	1	Input	SERDES Clock. Must be 4x the system clock.
i_reset	1	Input	System reset. Active high.
?_wb_ Wishbone Slv			Wishbone bus slave signals
?_dma_ Wishbone Master			DMA Wishbone bus master signals
o_ck	1	Output	SDIO/eMMC protocol clock signal
io_cmd	1	In/Out	SDIO/eMMC command wire
io_dat	NUMIO	In/Out	1, 4, or 8 SDIO/eMMC data lines
i_ds	1	Input	eMMC data strobe
i_card_detect	1	Input	High if the hardware detects a card, low o.w. Ignored if <code>OPT_CARD_DETECT</code> is not also set.
o_hwreset_n	1	Output	Low if requesting a hardware reset, high otherwise. Requires <code>OPT_HWRESET</code> be set.
o_1p8v	1	Output	High if requesting 1.8 Volt IO. Requires <code>OPT_1P8V</code> be set.
i_1p8v	1	Input	High if the interface is currently operating at 1.8 Volt IO, zero otherwise.
o_int	1	Output	An interrupt line to the CPU controller
o_debug	32	Output	See Verilog for details

Table 9.1: List of IO ports

Inputs and outputs are divided into sections: the master clock, the Wishbone bus, the SDIO/eMMC interface to the card, and four other wires. Of these, the last two chapters discussed the Wishbone bus interface and the clocks. The SDIO/eMMC interface should be fairly straightforward, so we'll move on and discuss the other four wires.

The `i_card_detect` wire should come from any card detection circuitry if present. This is an active high input. If `OPT_CARD_DETECT` is set, this wire will be used to set both the “Card not present” and “Card removed” bits when clear.

The eMMC standard includes a negative edge triggered hardware reset. If `OPT_HWRESET` is set, this reset is captured by `o_hwreset_n` and controlled by the `H` bit of the `CMD` register. This bit may also be used by SDIO devices as a reset control wire if desired.

The 1.8 Volt request line, `o_1p8v`, is used by the controller to indicate its desire to switch from 3.3 Volts to 1.8 Volts. The `OPT_1P8V` parameter should be set low if voltage switching is not supported. Likewise, the 1.8 Volt status line, `i_1p8v`, is used by software to know the current state of the 1.8 Volt switching. If set, it indicates that the IOs are operating at 1.8 Volts. This is to allow software to determine whether or not 1.8 Volt IOs are enabled, independent of whether or not `OPT_1P8V` indicates the IO voltage can be adjusted. Hence, if `OPT_1P8V` is not set, this value should be a constant reflecting the IO value expected.

The interrupt line, `o_int`, has been discussed in Chapt. 6. This line is synchronous with the system clock domain. Most interrupts are only one clock period wide in duration.

The final bus of 32-wires, `o_debug`, is defined internally and used when/if necessary to debug the core and watch what is going on within the design. When used, they are to be connected to an internal logic analyzer (ILA) within the FPGA. These wires may be left unconnected in most implementations, as they are not necessary for using the actually controller.

The final IO ports are the AXI stream ports used by the DMA. These are listed in Tbl. 9.2. These ports fall into two groups: the slave port, for receiving data into the controller for transport

Port	Width	Direction	Description
s_valid	1	Input	AXI Stream slave port <code>TVALID</code> signal. Ignored if <code>!OPT_STREAM</code> .
s_ready	1	Output	AXI Stream slave <code>TREADY</code> signal. Fixed at zero if <code>!OPT_STREAM</code> .
s_data	SW	Input	AXI Stream slave <code>TDATA</code> bits. Ignored if <code>!OPT_STREAM</code> .
m_valid	1	Output	AXI Stream master port <code>TVALID</code> signal. Fixed at zero if <code>!OPT_STREAM</code> . If set, then valid data is present and waiting to be transferred.
m_ready	1	Input	AXI Stream master <code>TREADY</code> signal. Controls backpressure within the IP, limiting the output IO rate.
m_data	SW	Output	AXI Stream master port <code>TDATA</code> signal.
m_last	1	Output	AXI Stream master port <code>TLAST</code> signal. Set on the last beat of every block (sector).

Table 9.2: AXI-Stream IO ports

to the SD card/eMMC, and the master port, for sending data read by the SD/eMMC to external components within the design.